

PySpark for Data Engineering — Complete Notes + Interview Q&A

Background assumed: Python + SQL, new to Spark **Practice environment:** Local PySpark (Databricks/cloud from Module 7 onward)

This guide covers all 9 modules of the roadmap. Each topic has: a concise concept explanation, then interview questions with answers (mix of conceptual, "why", and gotcha-style questions actually asked in DE interviews).

Module 1 — Spark Fundamentals

1.1 What Spark is and why it exists (vs. pandas, vs. Hadoop MapReduce)

Notes

Spark is a **distributed, in-memory data processing engine**. It splits data into partitions spread across a cluster of machines and processes them in parallel.

vs. pandas: pandas runs on a single machine, single-threaded (mostly), and everything must fit in RAM on that one machine. Spark distributes data + computation across many machines, so it scales to terabytes/petabytes. For data that fits comfortably in memory on one machine, pandas is faster and simpler — Spark has overhead (JVM startup, task scheduling, serialization) that doesn't pay off at small scale.

vs. Hadoop MapReduce: MapReduce writes intermediate results to disk between every map and reduce step, which is slow. Spark keeps intermediate data in memory (RDDs/DataFrames cached in RAM) whenever possible, making iterative and multi-step jobs 10-100x faster. Spark also offers a much richer, higher-level API (DataFrames, SQL, joins, window functions) versus MapReduce's low-level key-value map/reduce functions.

Spark unifies batch, streaming, SQL, ML (MLlib), and graph (GraphX) processing under one engine.

Interview Q&A

Q: Why would you choose Spark over pandas for a task? A: When the dataset doesn't fit in a single machine's memory, or when the job needs to run in parallel across a cluster to finish in reasonable time. Also when you need built-in fault tolerance, or need to read directly from distributed storage (S3, HDFS) at scale. For small data (fits in RAM, single machine), pandas is usually simpler and faster due to lower overhead.

Q: What's the core reason Spark is faster than MapReduce for multi-stage jobs? A: Spark keeps intermediate results in memory across stages (via RDD/DataFrame lineage and caching) instead of writing to disk after every map/reduce step like classic MapReduce does. This drastically cuts I/O for iterative algorithms and multi-step pipelines.

Q: Is Spark always faster than pandas? A: No. For small datasets that fit on one machine, Spark's distributed overhead (task scheduling, serialization, JVM/Python interop) can make it slower than pandas. Spark's advantage shows up at scale.

Q: What problem does Spark solve that a single powerful machine with lots of RAM couldn't? A: Even a single huge machine has a ceiling, and it's a single point of failure — if it dies, the job dies. Spark scales horizontally (add more machines) instead of vertically, and provides fault tolerance: if a node fails, Spark recomputes lost partitions from lineage rather than failing the whole job.

1.2 Cluster architecture: Driver, Executors, Cluster Manager

Notes

Driver: the process running your `main()`/script. It builds the DAG of transformations, negotiates resources with the cluster manager, schedules tasks, and collects results. There is exactly one driver per application.

Executors: worker processes on cluster nodes that actually run tasks (the real computation) and store data for caching. Each executor has a fixed number of cores and a memory allocation, and it runs tasks in parallel threads (one task per core slot).

Cluster Manager: allocates resources (CPU/memory) across applications running on the cluster. Options: **Standalone** (Spark's own simple manager), **YARN** (Hadoop ecosystem), **Kubernetes**, **Mesos** (largely deprecated now).

Flow: Driver asks Cluster Manager for executors → Cluster Manager launches executor processes on worker nodes → Driver sends tasks to executors → executors run tasks and report status/results back to driver.

Task, Stage, Job hierarchy: an **action** triggers a **Job** → a job is split into **Stages** at shuffle boundaries → each stage is split into **Tasks**, one per partition, run in parallel by executors.

Interview Q&A

Q: What happens if the driver dies? A: The whole application fails — the driver holds the application state, the SparkContext, and coordinates everything. There's no automatic driver failover in most deployment modes (client mode); in cluster mode with supervision, some managers can restart the driver, but any in-flight state is lost.

Q: What happens if an executor dies mid-job? A: Spark detects the failure, and the driver reschedules the lost tasks on other executors. Any cached data on that executor is lost and will be recomputed from lineage (RDD DAG) if needed again. This is Spark's fault tolerance model — no data replication needed, just recomputation from the DAG.

Q: Client mode vs. cluster mode — what's the difference? A: In **client mode**, the driver runs on the machine that launched the job (e.g., your laptop or an edge node) — good for interactive work but if that machine disconnects, the job dies. In **cluster mode**, the driver runs inside the cluster itself, submitted and managed by the cluster manager — more robust for production jobs, no dependency on the submitting machine staying alive.

Q: How does Spark decide how many tasks to run in parallel? A: Parallelism is bounded by the number of partitions in the stage (one task per partition) and the number of available executor cores (each core runs one task at a time). If you have 200 partitions but only 20 total cores across executors, only 20 tasks run concurrently; the rest queue.

Q: What's the difference between an executor and a core/slot? A: An executor is a JVM process on a worker node with allocated memory and a number of cores. Each core is a slot that can run one task at a time — so an executor with 4 cores can run 4 tasks concurrently.

1.3 SparkSession, SparkContext

Notes

SparkContext is the original entry point (Spark 1.x) — connection to the cluster, used to create RDDs, accumulators, broadcast variables.

SparkSession (Spark 2.0+) is the unified entry point that wraps SparkContext plus SQLContext/HiveContext — you use it for DataFrame/SQL work, which is basically all modern Spark code. You create it once per application:



```
python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()
```

`getOrCreate()` returns an existing session if one exists in the process (important in notebooks — avoids creating duplicate sessions).

You can still get the underlying `SparkContext` via `spark.sparkContext` if you need RDD-level APIs.

Interview Q&A

Q: Why did Spark introduce `SparkSession` if `SparkContext` already existed? A: Pre-2.0, users had to juggle `SparkContext`, `SQLContext`, `HiveContext`, and `StreamingContext` separately depending on what API they wanted. `SparkSession` unifies all of that into a single entry point, simplifying configuration and initialization.

Q: What does `getOrCreate()` do and why does it matter? A: It reuses an existing `SparkSession` in the current JVM process if one exists, otherwise creates a new one. This matters because Spark doesn't support multiple active `SparkContexts` in one JVM by default — calling `SparkSession.builder.getOrCreate()` repeatedly (e.g., in notebook cells) won't error out or create wasteful duplicate sessions.

Q: Can you have multiple `SparkSessions` in one application? A: You can create multiple `SparkSession` objects that share the same underlying `SparkContext` (via `spark.newSession()`), useful for isolating temp views/configs between logical units of work. But there's still only one `SparkContext` per JVM.

Q: How do you access RDD-level APIs from a `SparkSession`? A: `spark.sparkContext` gives you the underlying `SparkContext`, e.g. `spark.sparkContext.parallelize([1,2,3])`.

1.4 Lazy evaluation, transformations vs. actions

Notes

Transformations (e.g., `select`, `filter`, `withColumn`, `groupBy`, `join`) are **lazy** — they don't execute immediately. They just build up a logical plan (a DAG of operations).

Actions (e.g., `show()`, `collect()`, `count()`, `write()`, `take()`) trigger actual execution — Spark optimizes the accumulated plan and runs it across the cluster.

Why lazy evaluation matters: Spark can look at the *entire* chain of transformations before running anything, and optimize the whole thing (predicate pushdown, combining filters, choosing join strategies) via the Catalyst optimizer — instead of executing each line eagerly and missing optimization opportunities.

Common transformations are split into **narrow** (each output partition depends on one input partition — e.g. `filter`, `map`, `select`) and **wide** (output partitions depend on multiple input partitions, requiring a **shuffle** — e.g. `groupBy`, `join`, `distinct`, `repartition`).

Interview Q&A

Q: Why does Spark use lazy evaluation instead of executing each transformation immediately? A: It lets Spark build the full logical plan before running anything, then optimize the whole pipeline as a unit (e.g., pushing filters down before a join, pruning unused columns, choosing the best join strategy) rather

than executing sub-optimal steps one by one. It also avoids unnecessary computation on data that's filtered out later.

Q: If I write `df.filter(...).select(...).show()`, when does anything actually happen on the cluster? A: Nothing runs until `.show()` (the action) is called. `filter` and `select` just add nodes to the logical plan.

Q: What's the difference between a narrow and a wide transformation, and why does it matter? A: Narrow transformations (`map`, `filter`, `select`) can be computed independently per partition with no data movement between nodes. Wide transformations (`groupBy`, `join`, `repartition`) require data with the same key to be brought together, causing a **shuffle** — data is written to disk and transferred across the network between stages. Shuffles are expensive, so minimizing wide transformations (or the data volume going into them) is a key performance lever.

Q: What triggers a new Stage in a Spark job? A: A shuffle boundary. Every time a wide transformation requires repartitioning data, Spark ends the current stage and starts a new one after the shuffle.

Q: Give an example where lazy evaluation actually changes what work gets done, not just when. A: `df.filter(col("year") == 2023).select("id").show()` — if `df` is read from partitioned Parquet files partitioned by year, Catalyst can push the filter down to the file-scan level and skip reading files for other years entirely (partition pruning), and it can prune columns not selected. If it executed eagerly line by line, it might read the full dataset before applying the filter.

1.5 DataFrames vs. RDDs (why DataFrames win for data engineering)

Notes

RDD (Resilient Distributed Dataset): the low-level Spark abstraction — a distributed collection of objects with no schema awareness. You operate on it with functional operations (`map`, `filter`, `reduce`) using arbitrary Python/Scala functions.

DataFrame: a distributed collection of data organized into named columns with a schema (like a table). Built on top of RDDs, but with structure.

Why DataFrames win for data engineering:

Catalyst optimizer understands DataFrame operations (schema + relational operators) and can optimize the query plan. RDDs are opaque black-box functions — Spark can't optimize inside a Python lambda.

Tungsten execution engine uses off-heap binary memory format for DataFrames, avoiding JVM object overhead and Java garbage collection costs that RDDs incur.

DataFrames support SQL directly, and are far more concise for typical ETL (joins, aggregations, filtering) than hand-written RDD transformations.

Cross-language consistency — a DataFrame operation performs the same whether written in Python, Scala, or SQL, because it compiles down to the same optimized physical plan. RDD operations in PySpark are slower than Scala RDDs due to Python-JVM serialization overhead; this gap mostly disappears with DataFrames since the heavy lifting happens in the JVM regardless of the driving language.

RDDs are still useful for: fine-grained control over partitioning/physical distribution, unstructured data, or custom low-level algorithms that don't map well to relational operators.

Interview Q&A

Q: Why are DataFrames generally faster than RDDs for the same task in PySpark specifically? A: In PySpark, RDD transformations that use Python lambdas require moving data out of the JVM into a Python process for every partition, running the function, and sending results back — this serialization overhead is significant. DataFrame operations are expressed as relational operators; Spark compiles them into an

optimized physical plan (via Catalyst) that runs directly in the JVM with Tungsten's efficient binary memory format, so there's no per-row Python round-trip for the built-in operations.

Q: When would you drop down to RDDs from DataFrames? A: When you need very custom partitioning logic, need to process unstructured/complex data that doesn't fit a tabular schema well, need low-level control over data placement (e.g., custom partitioners for a specific algorithm), or are implementing something like a custom graph algorithm that doesn't map cleanly to relational operations.

Q: What is Catalyst and why can't it optimize RDD code? A: Catalyst is Spark's query optimizer — it takes the logical plan of DataFrame/SQL operations (which are declarative, like "select these columns, filter on this condition, join on this key") and applies optimization rules (predicate pushdown, constant folding, join reordering, etc.) before generating a physical execution plan. RDD transformations are arbitrary user-defined functions (Python/Scala closures) — Spark has no visibility into what's happening *inside* the function, so it can't reorder, push down, or optimize around it. It's a black box to the optimizer.

Q: What is Tungsten? A: Spark's project for CPU and memory efficiency — it uses off-heap binary memory management (avoiding Java object overhead and GC pressure), whole-stage code generation (compiling multiple operators into a single optimized JVM bytecode function), and cache-friendly data layouts. DataFrames benefit from Tungsten directly; RDDs of generic Java/Python objects don't.

Q: Are RDDs deprecated? A: No, but they're a lower-level API now mostly used internally or for edge cases. Official guidance is to use DataFrames/Datasets for anything that maps to structured/relational processing, which is the vast majority of data engineering work.

Module 2 — DataFrame Core Operations

2.1 Creating DataFrames (from files, from Python data, with schemas)

Notes

From files: `spark.read.csv(path)`, `.json(path)`, `.parquet(path)`, `.format("...").load(path)`.

From Python data:

```
python
data = [(1, "Alice"), (2, "Bob")]
df = spark.createDataFrame(data, ["id", "name"])
```

Also from pandas: `spark.createDataFrame(pandas_df)`.

With explicit schema (recommended for production over schema inference):

```
python
from pyspark.sql.types import StructType, StructField, IntegerType, StringType

schema = StructType([
    StructField("id", IntegerType(), nullable=False),
    StructField("name", StringType(), nullable=True),
])
df = spark.read.schema(schema).csv(path)
```

Schema inference (`inferSchema=True` for CSV, automatic for JSON/Parquet) is convenient but costs an extra read pass over the data (for CSV) and can guess wrong types (e.g., a column that's mostly ints but has one string value gets inferred as string, or numeric precision gets lost).

Interview Q&A

Q: Why prefer explicit schemas over `inferSchema=True` in production pipelines? A: Inference requires an extra pass over the data (for CSV specifically) to sample and guess types, which costs time on large files. It's also fragile — bad or unexpected values can silently change the inferred type across runs, breaking downstream code. Explicit schemas are faster, deterministic, and fail fast (or coerce predictably) on malformed data.

Q: What happens if incoming CSV data doesn't match your explicit schema? A: Depends on the mode: default `PERMISSIVE` mode sets unparseable fields to `null` and can put the raw malformed record into a `_corrupt_record` column if you include one in the schema. `DROPMALFORMED` drops bad records entirely. `FAILFAST` throws an exception immediately on a mismatch.

Q: How do you create a DataFrame from an existing pandas DataFrame, and what's a caveat? A: `spark.createDataFrame(pandas_df)`. Caveat: this collects/serializes all the pandas data and distributes it — fine for small data (test fixtures, lookup tables) but not meant for large datasets, since the entire pandas DataFrame must already exist in the driver's memory before conversion.

2.2 select, filter, withColumn, drop, rename

Notes




```
python
df.select("id", "name")           # choose columns
df.select(col("id"), (col("age") + 1).alias("age_plus_one"))
df.filter(col("age") > 18)         # same as .where(...)
df.withColumn("age_plus_one", col("age") + 1) # add/replace a column
df.drop("age")                    # remove a column
df.withColumnRenamed("age", "user_age")    # rename a column
```

`filter` and `where` are exact aliases — use whichever reads better.

`withColumn` either adds a new column or **replaces** an existing one if the name matches — this is a common source of bugs (accidentally overwriting a column you meant to keep).

Chaining many `withColumn` calls used to have a real performance cost in older Spark versions due to how the query plan analyzer handled each one (deep expression trees); modern Spark (3.x+) with `withColumns` (plural, accepts a dict) mitigates this, but it's still good practice to combine transformations where reasonable.

Interview Q&A

Q: What's the difference between `filter` and `where` in PySpark? A: None functionally — `where` exists as a SQL-like alias for `filter`. Pick based on team convention/readability.

Q: If you call `df.withColumn("age", col("age") * 2)`, what happens to the original `age` column? A: It's replaced — `withColumn` overwrites any existing column with the same name rather than erroring or adding a duplicate. If you wanted to keep both, you'd need a different alias, e.g. `.withColumn("age_doubled", col("age") * 2)`.

Q: Why can chaining dozens of `.withColumn()` calls be a performance problem, and how do you avoid it?

A: Historically, each `withColumn` call added to the logical plan, and with very large chains this could slow down query *analysis* (not necessarily execution) due to how the Catalyst analyzer processed the growing expression tree — this showed up as slow planning time on jobs with 100s of `withColumn` calls. The fix is to use `df.select(*[...all columns and new expressions...])` in one shot, or `df.withColumns({...})` (plural, dict-based, added in Spark 3.3) to add multiple columns in a single call.

Q: How do you select all columns except one or two, without listing every column name? A:

`df.drop("col_to_exclude")`, or `df.select([c for c in df.columns if c not in ("col1", "col2")])`.

2.3 Aggregations: groupBy, agg

Notes

```
python
df.groupBy("department").count()
df.groupBy("department").agg(
    F.avg("salary").alias("avg_salary"),
    F.max("salary").alias("max_salary"),
    F.countDistinct("employee_id").alias("headcount"),
)
df.groupBy("department", "team").agg(F.sum("revenue")) # multi-key grouping
```

`groupBy(...).count()` is a shortcut for a single aggregation; `.agg(...)` lets you compute multiple aggregates at once, which is far more efficient than calling multiple separate `groupBy` actions (each of which would be a separate job/shuffle).

`groupBy` is a wide transformation — it causes a shuffle to bring matching keys together (unless the data is already partitioned by the grouping key, or a partial pre-aggregation is possible — see AQE/combiner behavior below).

Spark performs partial aggregation on each partition **before** the shuffle (similar to a MapReduce combiner) for associative/algebraic functions like `sum`, `count`, `avg` — this reduces the amount of data shuffled significantly.

Interview Q&A

Q: Why is `df.groupBy("dept").agg(F.avg("salary"), F.max("salary"))` better than calling `.agg()` twice separately? A: A single `.agg()` call with multiple aggregate expressions computes everything in one pass/one shuffle. Calling `groupBy+agg` twice separately triggers two full shuffles over the data — twice the network and disk I/O for what could be one job.

Q: Does `groupBy` always cause a shuffle? A: In the general case, yes, because rows with the same key can live on any partition and need to be brought together. Spark does a partial/local aggregation on each partition first to reduce data volume before shuffling. If the data is already bucketed/partitioned by the grouping key, Spark can sometimes avoid or minimize the shuffle.

Q: What's the difference between `count()` and `countDistinct()` performance-wise? A: `count()` is a simple row count, cheap. `countDistinct()` needs to track unique values which requires more shuffle data (or a hash-based structure) and is more expensive, especially on high-cardinality columns. For approximate distinct counts at scale, `approx_count_distinct()` (HyperLogLog-based) trades a small error margin for much better performance.

Q: How would you compute multiple percentiles per group efficiently? A: Use

`F.expr("percentile_approx(col, array(0.5, 0.9, 0.99))")` inside `.agg()` — it's an approximate, single-pass, shuffle-efficient algorithm, versus exact percentile calculations which require sorting/collecting full distributions.

2.4 Sorting, distinct, limit

Notes



```
python
df.orderBy("age")           # ascending by default
df.orderBy(col("age").desc())
df.sort("dept", "age")      # alias for orderBy
df.distinct()               # dedupe all columns
df.dropDuplicates(["email"]) # dedupe on specific columns (keeps one row per key, arbitrary which)
df.limit(10)
```

`orderBy()`/`sort` is a **global** sort — it requires a shuffle to bring all data to be compared/ordered across partitions (unlike sorting within a partition).

`distinct()` internally works like a `groupBy` on all columns — also a shuffle.

`limit(n)` is comparatively cheap: Spark can often stop early once it has enough rows without processing all partitions fully, especially combined with `orderBy()` (`orderBy(...).limit(n)` gets optimized into a more efficient top-N operation rather than a full global sort).

Interview Q&A

Q: Why is a global `orderBy` on a huge DataFrame expensive, and what should you consider before using it?

A: It requires a full shuffle to physically bring together rows from all partitions in sorted order across the cluster (range partitioning) — comparable cost to a `groupBy`. If you only need sorted output within a downstream step (like a top-N per group), consider window functions or `orderBy().limit(n)` instead of sorting the entire dataset, since Spark can optimize the limit case (top-K) more cheaply than a full sort.

Q: What's the difference between `distinct()` and `dropDuplicates(["col"])`? A: `distinct()` treats the entire row (all columns) as the dedup key. `dropDuplicates(subset)` dedupes based on only the specified columns, keeping one arbitrary row per key (not guaranteed which one, unless you sort first).

Q: How do you get the row with the max value per group instead of just the max value? A: A

`groupBy().agg(F.max(...))` only gives you the max *value*, not the full row. Use a window function instead:

`Window.partitionBy("group").orderBy(col("value").desc())` With `row_number() == 1`, or Use `F.max_by("other_col", "value")` (Spark 3.0+) inside `.agg()` to directly get the associated column value at the max.

2.5 Joins (inner, left, right, outer, semi, anti) and join strategies

Notes

Join types:

`inner` — only matching keys in both sides.

`left` / `left_outer` — all rows from left, matched rows from right (nulls if no match).

`right` / `right_outer` — all rows from right, matched from left.

`outer` / `full` — all rows from both, nulls where unmatched.

`left_semi` — rows from left that **have** a match in right, but only left's columns (like a filter using a subquery — no duplication if right has multiple matches).

`left_anti` — rows from left that **do NOT** have a match in right (opposite of semi — great for "find records with no matching entry").



python

```
df1.join(df2, on="id", how="inner")  
df1.join(df2, df1.id == df2.user_id, how="left")  
df1.join(df2, on="id", how="left_semi")  
df1.join(df2, on="id", how="left_anti")
```

Join strategies (how Spark physically executes a join):

Broadcast Hash Join: if one side is small enough (default threshold `spark.sql.autoBroadcastJoinThreshold` = 10MB), Spark sends a full copy of the small table to every executor, avoiding a shuffle of the large table entirely. Fastest option when applicable.

Shuffle Hash Join: both sides are shuffled by join key, then a hash table is built on one side per partition. Used when broadcast isn't feasible but one side is still meaningfully smaller.

Sort-Merge Join: default for large-large joins — both sides shuffled and sorted by join key, then merged. Most robust/general but most expensive (shuffle + sort on both sides).

Broadcast Nested Loop Join / Cartesian: used for non-equi joins or when no join key exists — very expensive, avoid when possible.

Interview Q&A

Q: What's the difference between `left_semi` and `inner` join? A: `left_semi` returns only columns from the left table, filtered to rows that have at least one match in the right table — it never duplicates left rows even if right has multiple matches for a key. `inner` returns columns from *both* tables and produces one output row per matching pair — so if the right table has 3 matches for a key, inner join produces 3 output rows for that left row, while `left_semi` produces exactly 1.

Q: When would you use `left_anti` join? A: To find rows in the left table that have no corresponding match in the right table — e.g., "find customers who placed an order but have no entry in the returns table," or validating referential integrity, or finding new records not yet in a target table (a common pattern in incremental/upsert pipelines).

Q: How does a broadcast join improve performance, and when would you force one? A: It avoids shuffling the large table entirely by sending a full copy of the small table to every executor — each executor can then do a local, shuffle-free join. You'd force one with `F.broadcast(smaller_df)` when Spark's optimizer doesn't detect the table is small enough (e.g., statistics aren't available, or it's just under/over the auto-broadcast threshold) but you know from context that it safely fits in executor memory.

Q: What's the danger of broadcasting a table that's actually too large? A: Each executor must hold the entire broadcast table in memory in addition to its normal workload — if it's too large, you get executor OOM errors. Broadcasting is only safe for genuinely small tables (rule of thumb: well under executor memory, typically under a few hundred MB to low GBs depending on cluster sizing).

Q: Why is sort-merge join the default for large-to-large joins instead of shuffle hash join? A: Sort-merge join is more memory-robust — it doesn't require building an entire hash table in memory for one side, it processes sorted streams, so it degrades more gracefully (spilling to disk) under memory pressure.

Shuffle hash join can be faster when it applies safely, but is riskier for OOM on skewed or large partitions, so Spark's optimizer favors sort-merge as the safe default for big-big joins.

Q: You join two DataFrames on a key with heavy skew (one key has 90% of rows). What breaks and how do you fix it? A: One task ends up processing a disproportionate share of the shuffle data (the skewed key), so that single task/partition runs far longer than the rest — the job appears "stuck" at 99% while one task grinds. Fixes: **salting** the skewed key (add a random suffix to spread it across more partitions and de-salt after aggregation), enabling **Adaptive Query Execution (AQE)** with skew join optimization (Spark 3.x automatically splits skewed partitions), or broadcasting the smaller side if applicable to avoid the shuffle on the skewed key altogether.

Module 3 — Reading & Writing Data

3.1 File formats: CSV, JSON, Parquet, ORC, Avro

Notes

Format	Type	Schema	Compression	Splittable	Best for
CSV	Row-based, text	No (inferred/manual)	Optional (gzip = not splittable)	Depends	Interop, human-readable, small/interchange data
JSON	Row-based, text	Semi (nested)	Optional	Depends	Semi-structured/nested data, APIs
Parquet	Columnar, binary	Embedded	Built-in (Snappy default)	Yes	Analytics, DE pipelines — Spark's default/preferred
ORC	Columnar, binary	Embedded	Built-in	Yes	Hive-heavy ecosystems, similar to Parquet
Avro	Row-based, binary	Embedded (schema evolution friendly)	Built-in	Yes	Streaming/Kafka payloads, schema evolution

Parquet is the de facto standard for Spark data engineering: columnar storage means Spark reads only the columns actually needed (**column pruning**), and it stores per-column statistics (min/max) that enable **predicate pushdown** (skipping row groups that can't match a filter) — both dramatically reduce I/O for analytical queries that touch a subset of columns/rows.

Row-based formats (CSV/JSON/Avro) are better when you typically read/write *entire records* rather than a few columns — e.g., Avro is the standard for Kafka messages because you're serializing/deserializing whole events, not analytical column slices.

CSV/JSON without splittable compression (e.g., plain gzip) can become a bottleneck: a single gzip file can only be processed by one task, no matter how large the cluster is.

Interview Q&A

Q: Why is Parquet generally preferred over CSV for Spark data engineering pipelines? A: Parquet is columnar (Spark reads only needed columns — column pruning), stores schema and column-level min/max statistics inline (enabling predicate pushdown / row-group skipping), compresses better due to columnar layout with similar data types together, and is splittable, so multiple tasks can read a single file in parallel. CSV has none of these — no schema, no pushdown, and gzip-compressed CSV isn't splittable at all.

Q: Why is Avro often used for Kafka/streaming payloads instead of Parquet? A: Streaming systems deal with individual, whole records/events being written and read one at a time (not column-sliced analytical queries), so a row-based format is a more natural fit. Avro also has strong, well-supported schema evolution semantics (adding/removing fields safely with a schema registry) which matters a lot in long-running streaming pipelines where producers and consumers evolve independently.

Q: What does "splittable" mean and why does it matter? A: A splittable file can be divided into chunks that different tasks/executors process independently and in parallel. A non-splittable file (e.g., a single large gzip-compressed CSV) must be processed start-to-finish by one task — no matter how many cores your cluster has, that one file becomes a serial bottleneck. Parquet/ORC/Avro handle this natively via internal block structure; for CSV/JSON, you need a splittable compression codec (like bzip2) or just avoid heavy compression on files you need parallelized.

Q: What is predicate pushdown, concretely, for Parquet? A: Each Parquet file is split into row groups, and each row group stores min/max (and sometimes null count) statistics per column. When you filter (`df.filter(col("date") > "2024-01-01")`), Spark checks these stats and skips entire row groups where the filter can't possibly be true, without ever reading the actual data for those row groups — reducing I/O significantly, especially combined with partitioning.

3.2 Schema inference vs. explicit schemas (StructType)

Notes



```
python
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, TimestampType, ArrayType

schema = StructType([
    StructField("user_id", IntegerType(), False),
    StructField("name", StringType(), True),
    StructField("signup_ts", TimestampType(), True),
    StructField("tags", ArrayType(StringType()), True),
])

df = spark.read.schema(schema).json(path) # no inference pass needed
```

Parquet/Avro/ORC carry embedded schema — no inference needed, they're self-describing.

CSV/JSON schema inference requires Spark to **read the data once just to guess types** (a full or sampled pass), which is wasted work you pay every single run in production, and reruns can produce different results if the input data shape drifts.

Explicit schemas make pipelines: faster (skip inference pass), more predictable (fixed contract), and safer for data quality (mismatches surface immediately rather than silently coercing).

Interview Q&A

Q: What's the production risk of relying on `inferSchema=True` for a recurring pipeline reading daily CSV drops? A: If one day's file has an anomaly (e.g., a numeric column that happens to contain one blank/text value that day), the inferred type can flip (e.g., from `IntegerType` to `StringType`), silently changing downstream behavior — comparisons, joins, or aggregations that expected numeric types could break or

produce wrong results without any obvious error. An explicit schema makes the contract fixed and predictable, and bad values become nulls or trigger a defined failure mode instead of silently reshaping the schema.

Q: How do you handle a JSON source with deeply nested / evolving structure without hardcoding a massive schema by hand? A: Options: sample a representative subset and infer once, then persist that as a fixed schema definition (JSON schema file) reused across runs; use `spark.read.json(path, samplingRatio=...)` to control inference cost; or maintain the schema centrally (e.g., a schema registry) if the source is a stream, keeping it versioned deliberately rather than inferring per run.

3.3 Partitioned writes, write modes

Notes



python

```
df.write.mode("overwrite").partitionBy("year", "month").parquet(output_path)
```

Write modes: `append` (add to existing data), `overwrite` (replace existing data at the path), `errorIfExists` / `error` (default — fail if path exists), `ignore` (no-op if path exists).

`partitionBy` writes data into a directory hierarchy (`year=2024/month=01/...`), which enables **partition pruning** on read — a query filtering `WHERE year = 2024` only reads the relevant subdirectories, skipping the rest entirely.

Danger: over-partitioning (too many small partitions, e.g., partitioning by a high-cardinality column like `user_id`) creates the **"small files problem"** — thousands/millions of tiny files, which is slow for both writing (many small tasks) and reading (huge metadata overhead listing/opening files).

`overwrite` mode with `partitionBy` by default overwrites the **entire table/path**, not just the partitions being written — unless you set `spark.sql.sources.partitionOverwriteMode = "dynamic"`, which overwrites only the specific partitions present in the DataFrame being written (much safer for incremental jobs).

Interview Q&A

Q: What's the "small files problem" and how does partitioning cause it? A: When you partition by a high-cardinality column (or too many partition columns combined), you end up with a huge number of output directories, each containing very small files (sometimes one row's worth). This is bad because: (1) writing many small files has per-file overhead, (2) reading later requires opening/listing huge numbers of files, which is slow especially on object stores like S3 where listing is a network call, and (3) it wastes storage due to per-file overhead (footers, metadata). Fix: choose lower-cardinality partition columns (e.g., date, not `user_id`), and/or use `repartition()` / `coalesce()` before writing to control file count, or compact small files periodically.

Q: Difference between `overwrite` and dynamic partition overwrite mode — why does this matter for daily incremental loads? A: Static `overwrite` (default) replaces the *entire* output path/table when you write, even if your DataFrame only has data for one day — you'd wipe out all other days' data. Dynamic partition overwrite (`spark.sql.sources.partitionOverwriteMode=dynamic`) only overwrites the specific partitions present in the DataFrame (e.g., just `year=2024/month=06/day=15`), leaving all other partitions untouched — essential for safely re-running a single day's ETL without nuking the whole table.

Q: How do you control the number of output files per partition? A: `df.repartition(n, "partition_col").write.partitionBy("partition_col")...` — repartitioning before write controls how many

files land in each partition directory (each in-memory partition becomes roughly one output file). Too few = huge files, hard to parallelize reads; too many = small files problem.

3.4 Handling nested/semi-structured data (structs, arrays, maps)

Notes



python
Accessing struct fields
`df.select("address.city", "address.zip")`
`df.select(col("address").getField("city"))`
Exploding arrays into rows
`df.withColumn("tag", F.explode("tags"))` # one row per array element
`df.withColumn("tag", F.explode_outer("tags"))` # like explode, but keeps rows with null/empty arrays
Working with maps
`df.select(F.col("attributes")["color"])`
`df.withColumn("keys", F.map_keys("attributes"))`
Flattening / building structs
`df.withColumn("full_address", F.struct(col("city"), col("zip")))`
`df.select("*", "address.*")` # expand all struct fields as top-level columns

`explode` is a wide-ish operation in the sense that it changes row count (1 row → N rows), though it doesn't require a shuffle by itself — it's still worth being mindful of, since it can massively inflate row counts and downstream data volume.

`explode` drops rows where the array is null or empty by default; use `explode_outer` to preserve them (with null in the exploded column).

Nested schemas print with `df.printSchema()` showing indentation for structs/arrays — always inspect this before writing access code on unfamiliar nested JSON.

Interview Q&A

Q: You have a column of arrays and need one row per array element. What function, and what's the gotcha with empty arrays? A: `F.explode(col)` — but it silently drops the row entirely if the array is null or empty, which can quietly shrink your row count in ways that break downstream joins/counts. Use `F.explode_outer(col)` if you need to preserve those rows (producing a null in the exploded column instead).

Q: How do you flatten a deeply nested struct into top-level columns? A: `df.select("*", "struct_col.*")` expands all fields of a struct as top-level columns (and you can drop the original struct column after). For deeply nested structures, this is often done recursively — walking `df.schema` programmatically to build a full flattening `select` expression, since manually listing paths for deeply nested/evolving schemas doesn't scale.

Q: What's the performance concern with `explode` on a large dataset with big arrays? A: It multiplies row count (a row with a 1000-element array becomes 1000 rows), which can massively increase the volume of data flowing into every downstream operation — this can turn a modest dataset into a huge one,

causing spills, longer shuffles, and memory pressure in later stages, even though `explode` itself is a narrow, no-shuffle operation.

Q: How do you access a specific key from a MapType column? A: `df.select(col("map_col")["key_name"])` or `F.element_at(col("map_col"), "key_name")` (the latter is preferred since it lets you control null behavior for missing keys explicitly).

Module 4 — Spark SQL

4.1 Temp views, `spark.sql()`

Notes



```
python
df.createOrReplaceTempView("employees") # session-scoped, gone when session ends
df.createOrReplaceGlobalTempView("employees") # cross-session within the app, lives in "global_temp"

result = spark.sql("""
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
""")
```

Temp views let you mix SQL and DataFrame API freely — both compile down to the **same** logical/physical plan via Catalyst, so there's no performance difference between writing SQL vs. equivalent DataFrame code. Choice is purely about team/readability preference.

`createOrReplaceTempView` is session-scoped (disappears when the SparkSession ends);

`createOrReplaceGlobalTempView` is visible across sessions in the same Spark application, accessed via

`global_temp.viewname`.

A temp view is **not materialized data** — it's just a named reference to a logical plan. Querying it re-triggers the underlying computation (or reads underlying cache, if the DataFrame was explicitly `.cache()`'d).

Interview Q&A

Q: Is there a performance difference between writing Spark SQL vs. the equivalent DataFrame API code? A: No — both go through the same Catalyst optimizer and produce the same physical execution plan. The choice is stylistic: SQL is great for analysts/portability and complex declarative queries; the DataFrame API is better for programmatic composition, reuse, and integrating with regular Python control flow.

Q: What's the difference between a temp view and actually persisting/caching a DataFrame? A: A temp view is just a name pointing at a logical plan (like a saved query, not saved data) — every time you query it, Spark re-executes (or re-plans) the underlying transformations. `.cache()`/`.persist()` actually materializes and stores the computed data (in memory/disk) so repeated access doesn't redo the computation. You can combine both: cache a DataFrame, then register it as a temp view, so repeated SQL queries against that view hit the cached data.

Q: Why would you use `createOrReplaceGlobalTempView` instead of the regular one? A: When you need the view accessible across multiple SparkSessions within the same application (e.g., in a notebook

environment where you create a new session, or in multi-tenant style isolation but still want to share a specific view). It's rarer in typical single-session ETL jobs.

4.2 Window functions (rank, row_number, lead/lag, running totals)

Notes



```
python
from pyspark.sql.window import Window
import pyspark.sql.functions as F

w = Window.partitionBy("department").orderBy(F.col("salary").desc())

df.withColumn("rank", F.rank().over(w))
df.withColumn("dense_rank", F.dense_rank().over(w))
df.withColumn("row_number", F.row_number().over(w))
df.withColumn("prev_salary", F.lag("salary", 1).over(w))
df.withColumn("next_salary", F.lead("salary", 1).over(w))

# Running total
w_running = Window.partitionBy("department").orderBy("date").rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("running_total", F.sum("amount").over(w_running))
```

`rank()` leaves gaps after ties (1,1,3); `dense_rank()` doesn't (1,1,2); `row_number()` gives strictly unique sequential numbers regardless of ties (arbitrary order among ties, based on `orderBy` tiebreak or physical order if unspecified).

Window functions require data to be partitioned and sorted by the window spec — this involves a shuffle (by the `partitionBy` key) and a sort within each partition, similar cost profile to a `groupBy` but **rows are preserved** (no aggregation collapsing rows), which is the key difference from `groupBy`.

Common pattern — **top-N per group**: window with `row_number()`, then filter `WHERE row_number <= N`.

Interview Q&A

Q: What's the difference between `rank()`, `dense_rank()`, and `row_number()`? A: All three assign an ordering number within a window partition. `row_number()` always gives unique, sequential numbers even for tied values (ties broken arbitrarily/by additional order columns). `rank()` gives tied rows the same number, then skips the next rank(s) accordingly (e.g., 1,1,3). `dense_rank()` also ties equal rows but doesn't skip the next number (1,1,2).

Q: How would you find the top 3 highest-paid employees per department using Spark? A:



```
python
w = Window.partitionBy("department").orderBy(F.col("salary").desc())
result = df.withColumn("rn", F.row_number().over(w)).filter(F.col("rn") <= 3).drop("rn")
```

This is far more efficient than trying to do this with a self-join or multiple `groupBy`/max operations.

Q: What's the difference between using a window function vs. `groupBy` to compute an aggregate like average salary per department? A: `groupBy().agg()` collapses the data — one output row per group, losing individual row detail. A window function (`F.avg("salary").over(Window.partitionBy("department"))`) computes the aggregate but **keeps every original row**, attaching the group aggregate as an additional column to each row — useful when you need to compare an individual row's value to its group aggregate (e.g., "salary vs. department average") without a separate join back to the original data.

Q: What does `rowsBetween(Window.unboundedPreceding, Window.currentRow)` do, and how is it different from `rangeBetween`? A: It defines a running/cumulative window frame — from the start of the partition up to (and including) the current row, based on physical row position. `rangeBetween` instead defines the frame based on the *value* of the order-by column (e.g., all rows within a value range like "last 7 days" relative to the current row's date), which matters when there are duplicate/tied order values or you want a logical time window rather than a fixed row count.

Q: Are window functions expensive? What's the cost profile? A: They require a shuffle to bring each partition's rows together (by `partitionBy` key) plus a sort within the partition (by the `orderBy` clause) — comparable shuffle cost to a `groupBy`, but since no rows are dropped/aggregated away, the output volume stays as large as the input, and a poorly chosen partition key (or no partitioning at all, forcing everything into one window) can cause serious skew — e.g., `Window.orderBy("date")` with no `partitionBy` puts all data in a single partition for the window computation, which doesn't scale.

4.3 UDFs vs. built-in functions (and why to avoid UDFs when possible)

Notes

```
python
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType

@udf(returnType=IntegerType())
def add_one(x):
    return x + 1 if x is not None else None

df.withColumn("x_plus_1", add_one(col("x")))
```

Why avoid Python UDFs when a built-in function exists: built-in functions (`pyspark.sql.functions`) execute entirely inside the JVM with Catalyst/Tungsten optimization (whole-stage code generation). Python UDFs are **opaque black boxes** to Catalyst — no optimization inside them — AND they require row-by-row serialization of data from the JVM to a Python worker process and back, which is a major overhead, especially at scale.

Order of preference: (1) built-in `pyspark.sql.functions`, (2) Spark SQL expressions via `F.expr("...")` for things not directly exposed as a function, (3) Pandas UDFs (vectorized, see 4.4) if custom logic is truly needed, (4) plain Python UDFs as a last resort.

UDFs also lose type/null-safety guarantees and can be a debugging headache since errors surface inside the Python worker with less clear stack traces.

Interview Q&A

Q: Why are plain Python UDFs slow in PySpark compared to built-in functions? A: Two reasons: (1) Catalyst can't see inside a UDF — no predicate pushdown, no codegen, no optimization across it, it's a black box in the plan. (2) Execution-wise, Spark has to serialize each row's data out of the JVM, send it to a separate Python worker process, execute the Python function row-by-row (or in small batches), and serialize results back into the JVM — this cross-process/cross-language overhead dominates for anything non-trivial at scale.

Q: Give a concrete example of replacing a UDF with built-in functions. A: Instead of a UDF like `def categorize(age): return "minor" if age < 18 else "adult"`, use `F.when(col("age") < 18, "minor").otherwise("adult")` — same logic, but expressed with Catalyst-native conditional expressions that get optimized and code-generated like any other built-in.

Q: When is a UDF actually justified? A: When the logic genuinely can't be expressed via built-ins or `F.expr()` SQL expressions — e.g., calling a complex third-party Python library (regex library with unusual features, a specialized parsing/ML library), or truly bespoke business logic with many branches that would be unreadable as nested `when/otherwise`. Even then, prefer a **Pandas UDF** over a plain UDF for the performance win.

Q: What's a practical way to check if you actually need a UDF before writing one? A: Check `pyspark.sql.functions` docs/`F.expr()` (which exposes the full Spark SQL function library, including things not wrapped as Python functions) — Spark's built-in function set is large (string, date, array, map, math, JSON functions) and covers the vast majority of "I need a UDF" instincts coming from a pandas/Python background.

4.4 Pandas UDFs (vectorized UDFs)

Notes

```
python
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def add_one_vectorized(x: pd.Series) -> pd.Series:
    return x + 1
```

Pandas UDFs (aka vectorized UDFs) use **Apache Arrow** to transfer data between JVM and Python in efficient columnar batches, instead of row-by-row — and the Python function operates on a whole pandas Series/DataFrame at once (vectorized), rather than being called once per row.

This gives major speedups over plain UDFs (often 3-100x depending on the workload) because: (1) Arrow batch transfer avoids per-row serialization overhead, (2) the Python function itself runs vectorized pandas/numpy operations instead of a Python-level loop per row.

Types of Pandas UDFs: **Series to Series** (element-wise, most common), **Iterator of Series to Iterator of Series** (for cases needing expensive one-time setup, e.g. loading an ML model once per batch instead of per row), and **grouped map** (`applyInPandas` — apply an arbitrary pandas function to each group after a `groupBy`, useful for per-group custom logic that doesn't fit standard aggregations).

Still slower than pure built-in Spark functions (there's still a JVM↔Python boundary), but far better than row-at-a-time UDFs.

Interview Q&A

Q: How do Pandas UDFs achieve better performance than regular UDFs? A: They use Apache Arrow to move data between the JVM and Python processes in efficient columnar batches instead of row-by-row Python object serialization, and the UDF function itself receives a pandas Series/DataFrame and can use vectorized pandas/numpy operations instead of looping row by row in Python — both the transfer and the computation are batch/vectorized rather than per-row.

Q: When would you use `applyInPandas` (grouped map Pandas UDF)? A: When you need to run arbitrary, complex per-group logic that doesn't map to a standard Spark aggregate function — e.g., fitting a small statistical model per group, complex custom transformations using pandas/numpy/scipy per group, or anything where you conceptually want "for each group, run this whole pandas function and return a transformed DataFrame."

Q: Why would you use the "Iterator of Series" Pandas UDF variant instead of plain Series-to-Series? A: When there's expensive setup cost per UDF invocation — e.g., loading a large ML model or opening a network connection — you don't want to redo that for every batch. The iterator variant lets you do the expensive setup once, then process multiple batches (an iterator of them) through that same initialized state, amortizing the setup cost.

Q: Are Pandas UDFs as fast as native Spark built-in functions? A: No — there's still a JVM-Python process boundary and Arrow serialization cost, just far less than row-at-a-time UDFs. Built-ins run entirely in the JVM with full Catalyst optimization and no cross-process boundary at all, so they remain the fastest option when they can express the needed logic.

Module 5 — Performance & Internals

5.1 Catalyst optimizer & physical plans (`explain()`)

Notes

Catalyst pipeline: **Unresolved Logical Plan** (parsed from SQL/DataFrame calls, references not yet validated) → **Resolved (Analyzed) Logical Plan** (column/table references checked against catalog) → **Optimized Logical Plan** (rule-based optimizations: predicate pushdown, column pruning, constant folding, join reordering) → **Physical Plan(s)** (candidate execution strategies, e.g., which join algorithm) → **Cost model picks the best physical plan** → **Whole-stage code generation** compiles it to JVM bytecode for execution.

`df.explain()` shows the physical plan; `df.explain(True)` (or `explain("extended")`) shows all four stages (parsed, analyzed, optimized logical, and physical).

`df.explain("formatted")` (Spark 3.x) gives a more readable, structured plan output with numbered nodes.

Reading a plan: look for `PushedFilters` (predicate pushdown happened), `ReadSchema` (confirms column pruning — only needed columns listed), join type (`BroadcastHashJoin` vs `SortMergeJoin`), and `Exchange` nodes (= shuffle boundaries — count them, each is expensive).

Interview Q&A

Q: Walk through what happens when you call `.explain()` and see multiple "Exchange" nodes — what does that tell you? A: Each `Exchange` node represents a shuffle — a stage boundary where data is redistributed across the cluster (for a `groupBy`, join, repartition, or global sort). More Exchange nodes generally means more expensive shuffles and stage boundaries; you'd look at whether some of them are avoidable (e.g., could a join be a broadcast join instead, avoiding one Exchange; could two shuffles be combined by restructuring the query).

Q: How do you confirm predicate pushdown is actually happening for a filtered read on Parquet? A:

Run `df.filter(...).explain()` and check for a `PushedFilters` entry in the scan node — it lists the filter conditions that were pushed down to the file-scan/data-source level, meaning Spark skips row groups/files that can't match, rather than reading everything and filtering afterward in memory.

Q: What's the difference between the logical plan and the physical plan? A: The logical plan describes *what* the query wants (e.g., "join these two tables on this key, then filter"), independent of how it will be executed. The physical plan describes *how* Spark will actually execute it on the cluster (e.g., which join algorithm — broadcast vs sort-merge, how many partitions, in what order operators run) — Catalyst generates and cost-compares multiple candidate physical plans and picks one.

Q: What is whole-stage code generation and why does it help? A: Instead of executing each operator (filter, project, etc.) as a separate step with virtual function calls and intermediate object materialization (the "Volcano" iterator model), Spark's Tungsten engine fuses multiple operators within a stage into a single, specialized piece of JVM bytecode at runtime — reducing virtual call overhead and enabling CPU-level optimizations (like keeping data in CPU registers/cache), much closer to how a hand-written, specialized loop would perform.

5.2 Partitions, repartition vs. coalesce

Notes



python

```
df.repartition(200)          # full shuffle, can increase or decrease partitions, evens out data  
df.repartition(200, "key_col")  # shuffle + partition by a key (co-locates same key values)  
df.coalesce(50)              # no full shuffle, only merges existing partitions, can only decrease count
```

`repartition(n)`: triggers a full shuffle to redistribute data into exactly `n` partitions as evenly as possible.

Can increase or decrease partition count. Useful when you need to increase parallelism, fix skew, or partition by a specific key before a `groupBy`/`join` to co-locate matching keys.

`coalesce(n)`: merges existing partitions together **without a full shuffle** (data isn't reshuffled across the network — adjacent partitions are just combined). Can **only decrease** partition count, and can result in **uneven** partition sizes since it's not a true rebalance — it's cheap but not always a fair redistribution.

Common pattern: many small input partitions (small files) → `coalesce()` before writing to reduce output file count cheaply. If you need genuinely even distribution (e.g., before a heavy downstream computation), use `repartition()` despite the extra shuffle cost.

`spark.sql.shuffle.partitions` (default 200) controls how many partitions a shuffle (`groupBy`/`join`/etc.) produces — a very common tuning knob; too high wastes overhead on tiny tasks, too low under-utilizes the cluster/causes large partitions.

Interview Q&A

Q: When would you use `coalesce()` instead of `repartition()`, and what's the tradeoff? A: Use `coalesce()` when you only need to *reduce* the number of partitions (e.g., before writing output, to avoid the small-files problem) and don't need perfectly even distribution — it avoids a full shuffle, which is much cheaper. The tradeoff is that resulting partitions can be uneven in size, since it just merges existing partitions locally rather than truly rebalancing data across the cluster. If balance matters (e.g., you're about to run something expensive per partition and skew would hurt), use `repartition()` instead, accepting the shuffle cost.

Q: Why can't `coalesce()` increase the number of partitions? A: Because it only merges existing partitions together locally (without shuffling data across the network) — there's no mechanism to *split* a partition's data without moving/redistributing it, which is exactly what a shuffle does. To increase partition count you need `repartition()`, which does a full shuffle capable of splitting data into more, evenly-sized groups.

Q: What's `spark.sql.shuffle.partitions` and what happens if it's misconfigured? A: It's the number of partitions Spark uses for shuffle operations (joins, groupBys, etc.) — default is 200, a legacy default not tuned to your data size. Too high (relative to data size) creates many tiny partitions/tasks — overhead from task scheduling dominates actual work. Too low creates huge partitions — fewer, larger tasks, risking spills to disk or OOM, and under-utilizing available cluster parallelism. With Adaptive Query Execution (Spark 3.x), Spark can automatically coalesce shuffle partitions after the fact based on actual data size, reducing the need for manual tuning.

Q: You have 10,000 tiny output files after a job. What are your options to fix this? A: `coalesce(n)` right before the write to reduce file count cheaply (best if data's already reasonably balanced), or `repartition(n)` if you also need better balance across those files, or reconsider the partitioning strategy (`partitionBy` column choice) if the small files stem from over-partitioning on a high-cardinality column, or run a periodic compaction job downstream to merge small files after the fact.

5.3 Shuffles — what causes them, how to reduce them

Notes

Caused by: `groupBy`/`agg`, joins (except broadcast joins), `distinct`, `repartition`, global `orderBy`/`sort`, `window` functions with `partitionBy`.

Shuffle mechanics: each task writes shuffle output to local disk (**shuffle write**), partitioned by the target key's hash (or range for sorts); downstream tasks then read (**shuffle read**/fetch) the relevant partitions across the network from wherever they were written. This disk I/O + network transfer + serialization is why shuffles are the single biggest cost driver in most Spark jobs.

Ways to reduce shuffle cost:

Use **broadcast joins** for small-large joins to skip the shuffle on the large side entirely.

Filter and select early — reduce data volume *before* a shuffle-triggering operation (predicate/projection pushdown helps automatically, but also do it explicitly in your code for clarity and to guide Catalyst).

Avoid unnecessary `distinct()`/`repartition()` calls that aren't actually needed.

Pre-partition data at rest (e.g., bucketing, or write already partitioned by a common join/group key) so downstream shuffles can be reduced or skipped.

Combine multiple aggregations into one `.agg()` call rather than triggering multiple separate shuffles. Enable **Adaptive Query Execution (AQE)** to let Spark dynamically optimize shuffle partition sizes and join strategies at runtime based on actual data statistics.

Interview Q&A

Q: Name three Spark operations that always/typically cause a shuffle. A: `groupBy`/aggregations, joins that aren't broadcast joins (sort-merge/shuffle hash join), and `repartition()` — also `distinct()`, global `orderBy`, and window functions with `partitionBy`.

Q: What physically happens during a shuffle that makes it expensive? A: Each upstream task writes its output partitioned by the target key to local disk (shuffle write) — this is disk I/O. Then downstream tasks fetch the relevant partitions across the network from all the upstream executors (shuffle read/fetch) — this is network I/O plus serialization/deserialization overhead on both ends. This

disk+network+serialization combo, multiplied across every task, is why shuffles dominate job runtime for most non-trivial Spark jobs.

Q: How would you reduce shuffle volume in a job that joins a 500GB fact table with a 2GB dimension table? A: Broadcast the 2GB dimension table (if it comfortably fits in executor memory — may need to raise `autoBroadcastJoinThreshold` or explicitly call `F.broadcast()`) so the 500GB fact table never needs to be shuffled for the join at all — this is by far the biggest lever here.

Q: What's a shuffle spill, and why is it bad? A: When shuffle data being processed for a partition exceeds the available execution memory, Spark spills the excess to disk to avoid OOM — this adds disk I/O that wouldn't otherwise be needed, significantly slowing that task down. Frequent spills usually indicate skewed data, too few shuffle partitions relative to data volume, or insufficient executor memory, and show up clearly in the Spark UI's "Shuffle Spill" metrics.

5.4 Broadcast joins vs. shuffle joins

Notes

(See also 2.5.) Key performance framing here:

Broadcast join: no shuffle of the large table at all — the small table is sent once to each executor. Cost = network transfer of the small table × number of executors (once), plus memory to hold it on each executor. Best case for large-small join scenarios.

Shuffle joins (sort-merge / shuffle hash): both sides are shuffled across the network by join key. Cost scales with the size of *both* tables being shuffled — much more expensive at scale, but necessary when neither side is small enough to broadcast safely.

Tuning: `spark.sql.autoBroadcastJoinThreshold` (default 10MB) controls the size cutoff Spark uses to automatically choose a broadcast join. You can raise this (with caution re: executor memory) or force it explicitly via `F.broadcast(df)` when you know a table is safe to broadcast but Spark's size estimate (based on stats, which can be stale/missing) doesn't trigger it automatically.

Interview Q&A

Q: Why might Spark choose a sort-merge join even when you think a table is "small enough" to broadcast? A: Spark's decision relies on size estimates from table/file statistics, which can be stale, missing (e.g., no `ANALYZE TABLE` run), or simply exceed the configured `autoBroadcastJoinThreshold`. If Spark's estimate is off or conservative, it'll default to sort-merge. You can override this explicitly with `F.broadcast(df)`.

Q: What happens if you force a broadcast join on a table that turns out to be much larger than expected at runtime? A: Executors can run out of memory trying to hold and build a hash table from the (unexpectedly large) broadcast data, causing task/executor failures (often visible as `OutOfMemoryError` or broadcast timeout exceptions). This is why forcing broadcasts should only be done with real confidence in the data size, ideally validated, not just assumed.

Q: How does Adaptive Query Execution interact with join strategy selection? A: AQE can re-optimize the join strategy at runtime, after actual shuffle stage output sizes are known (rather than relying solely on pre-execution estimates) — e.g., it can convert a planned sort-merge join into a broadcast join mid-query if it observes that one side's actual materialized size, post-filter, turns out small enough. This makes broadcast join decisions more robust to inaccurate upfront statistics.

5.5 Caching/persistence strategies

Notes

```
python
df.cache() # shorthand for persist(MEMORY_AND_DISK) [Spark 3.x default]
df.persist(StorageLevel.MEMORY_ONLY)
df.persist(StorageLevel.MEMORY_AND_DISK_SER) # serialized, saves memory, costs CPU to deserialize
df.unpersist() # free up the cache
```

Caching only makes sense when a DataFrame will be **reused multiple times** (e.g., referenced in several downstream actions/branches) — caching something used only once adds overhead for no benefit, since the compute happens once either way and materializing it plus cache bookkeeping only helps if that same result is reused.

Caching is **lazy** too — calling `.cache()` doesn't materialize anything until the next action runs; that action both computes AND stores the result.

Storage levels: `MEMORY_ONLY` (fastest, but partitions that don't fit are simply dropped/recomputed on demand), `MEMORY_AND_DISK` (spills to disk instead of dropping — default for `.cache()`), `..._SER` variants (store data serialized — saves memory at the cost of CPU to deserialize on access), `..._2` suffix variants (replicate to 2 nodes for fault tolerance without recompute, at 2x storage cost).

Always `.unpersist()` (or let it fall out of scope, though explicit is safer) when a cached DataFrame is no longer needed, to free executor memory for other work — a common real-world bug is caching too much and causing memory pressure/eviction thrashing.

Interview Q&A

Q: When does caching a DataFrame actually help, and when is it wasted effort? A: It helps when the same DataFrame (or an intermediate result) is used multiple times downstream — e.g., you compute a filtered/joined DataFrame once and then branch into three different aggregations from it; caching avoids recomputing that shared upstream work three times. It's wasted when a DataFrame is used exactly once — you still pay the cost of computing it, plus caching overhead, with zero reuse benefit.

Q: What's the difference between `MEMORY_ONLY` and `MEMORY_AND_DISK` storage levels, and which does `.cache()` use? A: `MEMORY_ONLY` keeps partitions purely in RAM — any partition that doesn't fit is simply not cached (dropped) and gets recomputed from lineage whenever it's needed again. `MEMORY_AND_DISK` falls back to spilling excess partitions to local disk instead of dropping them, trading some disk I/O for guaranteed avoidance of recomputation. `.cache()` is shorthand for `persist(StorageLevel.MEMORY_AND_DISK)` in current Spark versions (it was `MEMORY_ONLY` historically in older Spark).

Q: What's a realistic failure mode from over-caching? A: Caching too many/too-large DataFrames without unpersisting old ones exhausts executor memory reserved for caching (`spark.memory.storageFraction`), causing eviction thrashing (Spark keeps evicting and recomputing cached partitions because there isn't room for everything), or worse, memory pressure that pushes into execution memory and causes spills/slowdowns or OOMs elsewhere in the job.

Q: Does `.cache()` guarantee data is materialized immediately? A: No — like all Spark operations, it's lazy. `.cache()` just marks the DataFrame to be cached; the actual computation and storage happens on the *next* action that touches it. A common gotcha is calling `.cache()` and then checking Spark UI storage tab before triggering any action — nothing will show up yet.

5.6 Skew handling (salting techniques)

Notes

Data skew: some key(s) have disproportionately more rows than others, so the task(s)/partition(s) handling those keys during a shuffle take far longer than the rest — the classic symptom is a Spark job "stuck" with most tasks done in seconds but one or two tasks running for many minutes.

Salting technique (manual fix):

Add a random "salt" suffix to the skewed key on one side of the join: `salted_key = concat(key, '_', rand_int(0, N))`.

Explode the *other* side of the join so each row is duplicated N times, once per possible salt value, matching the salted key range.

Join on the salted key — this spreads the previously-hot key's rows across N partitions instead of one.

Optionally strip the salt afterward if needed for output.

Alternative/modern fix: **Adaptive Query Execution (AQE)** in Spark 3.x has a built-in **skew join**

optimization (`spark.sql.adaptive.skewJoin.enabled`) that automatically detects skewed partitions (based on actual runtime shuffle statistics) and splits them into smaller sub-partitions handled by multiple tasks — largely eliminating the need to hand-roll salting for many common cases.

Other mitigations: isolate and handle the skewed key(s) separately (e.g., broadcast-join just the hot key's rows if that subset happens to be small, union with a normal join for the rest), or pre-aggregate before the join to shrink the skewed side.

Interview Q&A

Q: How do you detect that a slow Spark job is caused by data skew, specifically (vs. some other performance issue)? A: In the Spark UI, look at the Stages tab — a skewed stage shows very uneven task duration/shuffle-read-size distribution: most tasks finish quickly with small shuffle read sizes, while one or a handful of tasks have dramatically larger shuffle read size and much longer duration, dragging out the whole stage. This "long tail of one/few tasks" pattern is the signature of skew, as opposed to e.g. uniformly slow tasks (which points to something else, like insufficient overall parallelism or resource constraints).

Q: Explain salting in your own words, as if to someone who's never heard the term. A: Salting artificially breaks up an overloaded key into several fake sub-keys so the work for that one key gets spread across multiple partitions/tasks instead of piling onto one. You append a random number to the hot key on the big/skewed side (turning `key_A` into `key_A_0`, `key_A_1`, ... `key_A_N`), and duplicate the matching rows on the other side of the join once for every possible salt value, so every salted variant still finds its match. The join then distributes what used to be one giant partition into N more manageable ones.

Q: Does AQE's automatic skew handling mean you never need to think about skew manually anymore? A: Mostly for standard joins/shuffles, yes — AQE's skew join optimization handles the common case well by splitting oversized shuffle partitions automatically at runtime. But it doesn't cover every scenario (e.g., skew in a groupBy aggregation without a join, or cases where AQE's thresholds/config don't trigger for your specific data shape), so understanding manual salting is still valuable both for those edge cases and for interviews, since it demonstrates you understand *why* skew happens, not just which flag to flip.

Q: What's a cheap first check to see if skew is even a possibility before doing any of this? A:

`df.groupBy(join_key).count().orderBy(F.desc("count")).show()` — if one or a few keys have wildly more rows than the rest (e.g., a "null" or default/placeholder key absorbing a huge share, or one dominant customer/entity ID), that's your skew culprit, and you can target a fix at that specific key rather than guessing.

Module 6 — Data Engineering Patterns

6.1 Building idempotent, incremental ETL pipelines

Notes

Idempotent = running the same job twice (with the same input) produces the same end state, not duplicated/corrupted data. This matters because production jobs *will* be re-run — after failures, backfills, manual reruns — and a non-idempotent pipeline silently corrupts data on reruns.

Common idempotency patterns:

Overwrite by partition (dynamic partition overwrite, see 3.3) instead of blind `append` — reruns replace exactly the affected partition rather than duplicating rows.

MERGE/upsert (via Delta Lake `MERGE INTO`, see 6.5) — match on a natural key, update existing rows, insert new ones; reruns don't create duplicates.

Watermark/checkpoint tables — track "last successfully processed" timestamp/offset so a rerun knows exactly where to resume without reprocessing or skipping.

Deterministic transformations — avoid non-deterministic elements (e.g., `current_timestamp()` baked into business data, random IDs) unless intentionally tracked, since these make reruns produce different output even from identical input.

Incremental = processing only new/changed data since the last run, rather than reprocessing the entire dataset every time — essential once data volume grows past what a full daily reprocess can handle in your batch window.

Typically driven by a high-watermark column (e.g., `updated_at > last_checkpoint`) or CDC (change data capture) feed from the source system.

Interview Q&A

Q: What does "idempotent" mean in the context of an ETL pipeline, and why does it matter

operationally? A: An idempotent pipeline produces the same final state no matter how many times it's re-run with the same input — no duplicated rows, no corrupted aggregates. It matters because pipelines fail and get rerun constantly in production (transient cluster issues, upstream delays, manual backfills) — if a rerun isn't safe, every failure becomes a data-quality incident requiring manual cleanup, which doesn't scale operationally.

Q: Why is blind `append` mode dangerous for a daily batch job that might get rerun? A: If the job fails partway through, or gets manually rerun for the same day (e.g., after a bug fix), `append` mode just adds another full copy of that day's data on top of what's already there — silently duplicating every row. Dynamic partition overwrite or a MERGE-based upsert avoids this by replacing/reconciling exactly the affected data instead of blindly adding to it.

Q: How would you design an incremental load from a source table with no CDC feed, just an `updated_at` column? A: Maintain a checkpoint (e.g., in a small control table or config) recording the max `updated_at` successfully processed in the last run. Each run reads `WHERE updated_at > last_checkpoint`, processes it, and — critically — only advances the checkpoint *after* the write succeeds (so a failed run doesn't lose track of unprocessed data). You'd also need a strategy for late-arriving updates to old rows and potential clock skew at the source.

Q: What's a real risk with using `current_timestamp()` inside a transformation logic (not just as an audit column)? A: If `current_timestamp()` affects actual business logic (e.g., "flag as expired if now - created_at > 30 days" computed and stored as a boolean), a rerun days later produces a *different* result for the same input row than the original run did — breaking idempotency and reproducibility. Prefer computing against

a fixed, pipeline-run-parameterized reference date rather than a live wall-clock read, when reruns/backfills need consistent results.

6.2 Slowly Changing Dimensions (SCD Type 1 & 2) in Spark

Notes

SCD Type 1: overwrite the old value — no history kept. Simple: just update the row in place (or MERGE with `WHEN MATCHED THEN UPDATE`).

SCD Type 2: preserve full history — every change creates a new row, with the old row marked as no-longer-current (via an `is_current` flag and/or `effective_date`/`end_date` columns).



```
python
# SCD Type 2 pattern using Delta MERGE
from delta.tables import DeltaTable

target = DeltaTable.forPath(spark, "/path/to/dim_customer")

# Step 1: expire changed current rows
target.alias("t").merge(
    updates.alias("s"),
    "t.customer_id = s.customer_id AND t.is_current = true"
).whenMatchedUpdate(
    condition="t.address != s.address", # or any tracked attribute change
    set={"is_current": "false", "end_date": "current_date()"}
).execute()

# Step 2: insert new current rows for changed/new records
new_rows = updates.join(target.toDF(), "customer_id", "left_anti") \
    .withColumn("effective_date", F.current_date()) \
    .withColumn("end_date", F.lit(None)) \
    .withColumn("is_current", F.lit(True))
new_rows.write.format("delta").mode("append").save("/path/to/dim_customer")
```

Real implementations typically do this in a single well-orchestrated MERGE (Delta Lake supports multi-condition `whenMatched`/`whenNotMatched` clauses) rather than two separate steps, but the two-step logical breakdown above is how to *reason* about it.

Key columns for SCD2: `effective_date`, `end_date` (or `null` for current), `is_current` boolean, sometimes a surrogate key separate from the natural business key (since the same natural key now has multiple physical rows over time).

Interview Q&A

Q: What's the core difference between SCD Type 1 and Type 2, and when would you choose one over the other? A: Type 1 overwrites the old value with no history — use it when you don't need to know what a dimension attribute *used to be* (e.g., correcting a typo in a customer's name). Type 2 preserves full history by inserting a new row per change and marking old rows as expired — use it when historical accuracy

matters for reporting (e.g., "what region was this customer in when they placed this order" needs to reflect the region *at that time*, not today's region).

Q: Why do SCD Type 2 tables typically need a surrogate key separate from the natural/business key?

A: Because the same natural key (e.g., `customer_id`) now maps to multiple physical rows over time (one per historical version) — fact tables need to reference a *specific version* of the dimension row (the one that was current at the time of the fact event), which the natural key alone can't disambiguate. The surrogate key uniquely identifies each version-row.

Q: How do you detect which attributes actually changed, to decide whether a new SCD2 row is needed, versus a no-op?

A: Compare the incoming record's tracked attributes to the current row's values (typically the columns you care about tracking history for, not necessarily every column) — if any tracked attribute differs, it's a change requiring a new version; if everything matches, skip it as a no-op to avoid creating pointless new rows for unrelated/untracked field changes (like a `last_login_ts` that changes constantly but isn't a dimension attribute you're tracking history for).

Q: What's a common pitfall when implementing SCD2 with Spark/Delta if you're not careful about ordering?

A: If source updates for the same key arrive out of order (or multiple changes for the same key land in a single incremental batch), a naive MERGE can end up with an incorrect `effective_date`/`end_date` sequence or multiple rows marked `is_current = true` for the same key. You typically need to dedupe/order changes per key within the batch (e.g., using a window function to pick the latest state per key per batch) before merging into the target.

6.3 Data quality checks & validation

Notes

Categories of checks: **schema validation** (expected columns/types present), **completeness** (nulls in required fields, row counts within expected range), **uniqueness** (primary key duplicates), **referential integrity** (foreign keys exist in the parent table — checked via `left_anti` join), **freshness** (data isn't stale — max timestamp within expected recency), **business rule checks** (e.g., `amount >= 0`, `end_date >= start_date`).

Common tooling: **Great Expectations** (popular open-source DQ framework with expectation suites),

Deequ (AWS-built, Spark-native, computes data quality metrics at scale with statistical profiling), or hand-rolled checks using DataFrame operations + assertions integrated into the pipeline.

Where checks run: **pre-write gate** (validate before writing to prevent bad data reaching consumers — fail the job or quarantine bad records), or **post-write monitoring** (write anyway, but alert/track metrics — useful for softer/exploratory checks that shouldn't block a pipeline).

Pattern: split data into a "clean" and "quarantine" DataFrame based on validation rules, write the quarantine set to a separate location for investigation instead of silently dropping or blindly loading bad data.

Interview Q&A

Q: How would you check referential integrity between a fact table and a dimension table in Spark, without using a database's foreign key constraint?

A: `fact_df.join(dim_df, fact_df.dim_key == dim_df.dim_key, "left_anti")` — this returns fact rows whose key has **no** match in the dimension table, i.e., orphaned/invalid foreign keys. If this result is non-empty, you have a referential integrity violation to investigate or quarantine before proceeding.

Q: What's the difference between failing a pipeline outright on a data quality violation vs.

quarantining bad records? A: Failing outright is appropriate for critical, structural violations (e.g., schema

mismatch, primary key duplicates in a dimension) where continuing would corrupt downstream systems — better to stop and alert than propagate bad data. Quarantining (splitting off invalid rows to a separate location, loading the rest) is appropriate when a small percentage of bad records are expected/tolerable and blocking the entire pipeline over them would be overly disruptive — you still get visibility and can investigate/reprocess the quarantined set without holding up everything else.

Q: How would you check for unexpected row-count anomalies between pipeline runs (e.g., catching a source outage that returns a mostly-empty file)? A: Track historical row counts (e.g., in a metrics table) and compare each run's count against a rolling average/expected range (with some tolerance) — if today's count is, say, less than 50% of the trailing 7-day average, flag or fail the job rather than silently loading a suspiciously small (possibly broken/partial) dataset. This is a common and cheap "canary" check.

Q: What's Deequ, and why might you use it over hand-rolled DataFrame checks? A: Deequ is a Spark-native data quality library (originally from AWS/Amazon) that computes data quality metrics (completeness, uniqueness, distribution stats, anomaly detection) at scale using Spark's distributed engine, with a declarative API for defining expectation constraints. It's preferable over ad hoc checks when you need statistically robust profiling across large datasets efficiently, built-in anomaly detection over historical metric trends, and a standardized, reusable way to define/report data quality constraints across many pipelines rather than reinventing checks each time.

6.4 Medallion architecture (Bronze/Silver/Gold)

Notes

Bronze: raw data, ingested as-is from source (minimal/no transformation), often just schema-on-read applied, kept for full history/audit/reprocessing capability. Append-only typically.

Silver: cleaned, validated, conformed data — deduplicated, type-corrected, joined/enriched with reference data, data quality rules applied, schema enforced. Business-entity level (e.g., one clean `customers` table, one clean `orders` table).

Gold: aggregated, business-level, consumption-ready data — star schemas, pre-aggregated marts, feature tables for ML, dashboards' direct source tables. Optimized for specific downstream use cases (BI, reporting, ML).

Why layer this way: (1) **reprocessing safety** — if a bug is found in Silver logic, Bronze retains the untouched raw data so you can reprocess without re-extracting from the source system again; (2) **separation of concerns** — each layer has one job (capture, clean, serve); (3) **incremental trust** — data gets more reliable/structured as it moves up layers, and different consumers can plug in at the layer matching their trust/latency needs (a data scientist might read Silver directly for raw-ish exploration, a BI dashboard reads Gold).

Interview Q&A

Q: Why keep a Bronze layer at all instead of just cleaning data directly on ingestion? A: Bronze preserves the raw, unmodified source data, which is your safety net — if a bug is discovered in downstream cleaning/transformation logic, or business requirements change what "clean" means, you can reprocess from Bronze without needing to re-extract from the (possibly unavailable, rate-limited, or since-changed) original source system. It also serves as an audit trail of exactly what was received, which matters for debugging discrepancies and, in regulated industries, compliance.

Q: What kind of logic belongs in Silver vs. Gold? A: Silver: deduplication, type casting/schema enforcement, null/invalid record handling, joining with reference/lookup data to conform to a standard business entity definition — essentially "one clean, trustworthy version" of an entity (customers, orders,

products). Gold: business-specific aggregation and modeling — star schemas, KPI roll-ups, pre-joined wide tables tailored to a specific dashboard/ML feature set/reporting need. Silver is entity-centric and reusable across many downstream consumers; Gold is often use-case-specific and may be duplicated/denormalized differently for different consumers.

Q: If a data scientist needs near-raw data for exploratory analysis, which layer should they use, and why not Gold? A: Silver is usually the right fit — it's already deduplicated and type-safe (avoiding common raw-data pitfalls) but hasn't been aggregated or shaped for a specific business use case, so it retains full granularity and flexibility for exploration. Gold is typically pre-aggregated/pre-shaped for specific reporting needs and would lose the granularity or breadth a data scientist needs for open-ended exploration.

Q: Is medallion architecture specific to Delta Lake/Databricks, or a general pattern? A: It's a general data architecture pattern (multi-hop, staged refinement) that predates and is independent of any specific technology — you could implement it with plain Parquet on S3, a data warehouse's staging/core/mart schemas, etc. Databricks popularized the specific "Bronze/Silver/Gold" naming and it's commonly implemented with Delta Lake because Delta's ACID transactions and MERGE support make the incremental, reliable movement between layers much easier to implement correctly.

6.5 Delta Lake: ACID transactions, time travel, MERGE INTO, schema evolution

Notes

Delta Lake is a storage layer (built on Parquet + a transaction log) that adds database-like reliability features to data lake storage.

ACID transactions: every write to a Delta table is an atomic, all-or-nothing operation recorded in the `_delta_log` (a JSON/Parquet transaction log) — no more partial/corrupted writes from a failed job leaving the table in an inconsistent state (a real problem with plain Parquet, where a failed job could leave some files written and others not, with no atomicity).

Time travel: query a previous version of the table by version number or timestamp:

```
python
spark.read.format("delta").option("versionAsOf", 5).load(path)
spark.read.format("delta").option("timestampAsOf", "2024-01-15").load(path)
```

Useful for debugging ("what did this table look like before yesterday's bad job ran"), audits, and reproducing historical analyses.

MERGE INTO enables upserts in a single atomic operation (as used in SCD2 above):

```
sql
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Schema evolution: `.option("mergeSchema", "true")` on write allows new columns in the source to be automatically added to the target table schema, instead of failing on mismatch — useful for evolving

upstream sources, but should be used deliberately (unconstrained auto-evolution can let unexpected columns silently creep into a table).

Delta also enables `OPTIMIZE` (compacts small files into larger ones — directly addresses the small-files problem) and `VACUUM` (removes old, no-longer-referenced data files past a retention window, reclaiming storage — but this also limits how far back time travel can go).

Interview Q&A

Q: What real problem does Delta Lake's transaction log solve that plain Parquet-on-S3 doesn't? A:

Plain Parquet writes have no atomicity — if a Spark job writing many Parquet files fails partway through, you can be left with a partial write: some files present, others missing, no clear indication of what's "committed." Concurrent writers can also corrupt each other's output with no isolation. Delta's transaction log (`_delta_log`) records each write as an atomic, versioned commit — readers always see either the complete pre-write state or the complete post-write state, never a partial one, and concurrent writes are coordinated to prevent corruption.

Q: How would you use time travel to recover from a bad job that overwrote a table with incorrect data? A:

`spark.read.format("delta").option("versionAsOf", N).load(path)` (or `timestampAsOf`) to read the table as it was before the bad write, then write that data back as the current version (effectively a rollback) — e.g., `RESTORE TABLE my_table TO VERSION AS OF N` is an even more direct Delta SQL command for this. Note this only works within the table's retention window, since `VACUUM` eventually removes old file versions.

Q: What's the danger of using `mergeSchema=true` carelessly on every write? A: It lets any new column appearing in the source data silently get added to the target table's schema without explicit review — this can let unexpected/junk columns (from an upstream bug, a malformed record, or a schema change nobody was told about) creep into a production table unnoticed, gradually turning a clean, well-defined schema into an unmanaged sprawl. It's better used deliberately for known, intentional schema changes rather than left on by default everywhere.

Q: What does `OPTIMIZE` do, and how does it relate to the small-files problem? A: `OPTIMIZE` compacts many small files within a Delta table into fewer, larger files (using bin-packing), which directly addresses the small-files problem that accumulates from frequent small incremental writes (e.g., streaming micro-batches, frequent MERGE operations). This improves read performance significantly since there's far less per-file overhead (open/close, metadata) for subsequent queries. It can optionally be combined with `ZORDER BY` to also co-locate related data for better data skipping on specific query patterns.

Q: Why does `VACUUM` limit how far back you can time-travel? A: `VACUUM` physically deletes data files that are no longer referenced by the current table version and are older than the retention threshold (default 7 days) — this reclaims storage space, but it also means any Delta version that depended on those now-deleted files can no longer be reconstructed. Time travel and storage cost are in direct tension: longer retention = more time-travel history but more storage cost from retained old files.

Module 7 — Structured Streaming

7.1 Micro-batch vs. continuous processing

Notes

Spark Structured Streaming's default execution model is **micro-batch**: incoming data is grouped into small batches at a configurable trigger interval, and each micro-batch is processed using the *same* DataFrame/SQL engine as batch jobs (this is the key design idea — streaming and batch share one unified API and execution engine, unlike older Spark Streaming/DStreams which had a separate RDD-based model).

Continuous processing mode (experimental, more limited operator support) processes records as they arrive with much lower latency (~1ms vs ~100ms+ for micro-batch) but has significant restrictions on which operations it supports (no aggregations across most versions) and weaker fault-tolerance/exactly-once guarantees in some configurations — rarely used in practice compared to micro-batch.

Trigger types for micro-batch: default (run next batch as soon as previous one finishes),

`Trigger.ProcessingTime("10 seconds")` (fixed interval), `Trigger.Once()` / `Trigger.AvailableNow()` (process all currently available data then stop — used for scheduled/batch-like streaming jobs rather than always-on).

Interview Q&A

Q: Why did Spark unify streaming and batch processing under the same DataFrame API instead of keeping them separate? A: It lets you write the same transformation logic once and apply it to either batch or streaming data with minimal changes (often the only difference is `spark.read` vs

`spark.readStream`), reducing code duplication and cognitive overhead. It also means streaming jobs benefit from the same Catalyst/Tungsten optimizations as batch jobs, since under the hood a streaming query is just a series of incremental batch executions against the same query plan.

Q: What's `Trigger.AvailableNow()` used for, and how is it different from a continuously running streaming job? A: It processes all data currently available in the source (e.g., all new files/Kafka offsets present right now) and then stops, rather than running forever waiting for new data — effectively giving you streaming's incremental/checkpointed processing semantics but with batch-job-like scheduling (e.g., run via an orchestrator every hour). This is often preferred over an always-on streaming cluster when near-real-time latency isn't actually required, since it's more cost-efficient (cluster doesn't sit idle between scheduled runs).

Q: When would you actually need continuous processing mode instead of micro-batch? A: Only when you have a genuine sub-100ms end-to-end latency requirement that micro-batch's inherent batch-interval latency can't meet — this is a narrow, less common requirement in typical data engineering (vs. e.g. real-time fraud detection or similar). Given continuous mode's operator restrictions and more limited production maturity, most teams stick with micro-batch, tuning the trigger interval down as needed, unless there's a hard, proven latency requirement forcing the switch.

7.2 Reading streams (files, Kafka)

Notes



python

File source (e.g., new files landing in a directory)

```
stream_df = spark.readStream \
    .format("json") \
    .schema(schema) \
    .option("maxFilesPerTrigger", 10) \
    .load("/path/to/incoming/")
```

Kafka source

```
kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "broker1:9092,broker2:9092") \
    .option("subscribe", "my_topic") \
    .option("startingOffsets", "latest") \
    .load()
```

Kafka payload is binary — must deserialize the value column

```
parsed_df = kafka_df.select(
    F.from_json(F.col("value").cast("string"), schema).alias("data")
).select("data.*")
```

File streaming source **requires an explicit schema** (no inference on streaming sources — there's no stable "whole dataset" to sample from since it's unbounded).

Kafka source returns raw columns (`[key]`, `[value]`, `[topic]`, `[partition]`, `[offset]`, `[timestamp]`) as binary — you must explicitly cast/parse `[value]` (typically JSON or Avro-encoded) into a structured schema yourself.

`[startingOffsets]: "latest"` (only new messages from now on) vs `"earliest"` (replay from the beginning of retained topic data) — critical choice depending on whether you need full history or just new events going forward.

Interview Q&A

Q: Why can't you use `[inferSchema]` with a streaming file source the way you can with a batch read? A:

Schema inference needs to read (sample) the data to guess types, but a streaming source is unbounded/continuously arriving — there's no fixed, complete dataset to sample at query-start time. Spark requires an explicit schema for streaming sources so it knows how to parse each incoming micro-batch consistently from the start.

Q: What does Kafka's `[value]` column contain, and what do you have to do with it? A: It's raw binary (bytes) — Kafka itself doesn't know or enforce a schema for message payloads. You must cast it (`[.cast("string")]` for JSON payloads, or use an Avro/Protobuf deserializer for those formats) and then parse it into a structured DataFrame (e.g., `[F.from_json(col, schema)]`) yourself before you can work with it as structured fields.

Q: What's the practical difference between `[startingOffsets = "earliest"]` vs `"latest"`, and when would you pick each? A: `"earliest"` replays all retained messages in the topic from the beginning — use this for a new consumer that needs full historical context (e.g., backfilling a new downstream table) or when reprocessing after fixing a bug. `"latest"` only picks up messages produced from the moment the stream starts — use this for a consumer that only cares about new/ongoing events and doesn't need history (e.g., a live dashboard, alerting pipeline). Note this setting only matters on the *first* run without a checkpoint — once checkpointed, Spark resumes from the last committed offset regardless of this setting.

7.3 Watermarking, windowed aggregations

Notes



```
python
windowed_counts = stream_df \
    .withWatermark("event_time", "10 minutes") \
    .groupBy(
        F.window("event_time", "5 minutes"),
        "device_id"
    ) \
    .count()
```

Windowed aggregation: groups streaming data into time-based buckets (e.g., 5-minute windows) based on **event time** (when the event actually occurred, as recorded in the data) rather than **processing time** (when Spark happened to receive/process it) — event time is almost always the semantically correct choice for business logic (e.g., "orders per 5-minute window" should reflect when orders happened, not arbitrary processing delays).

Watermarking tells Spark how late data is allowed to arrive before being considered "too late" and dropped from aggregation — `withWatermark("event_time", "10 minutes")` means: once Spark has seen an event time up to T , it will wait for late data up to $T - 10min$ before finalizing/closing out that window's aggregation state and freeing its memory.

Watermarking exists to bound **unbounded state growth**: without it, Spark would have to keep aggregation state for every window open forever (since late data could theoretically arrive at any point), which would grow memory usage indefinitely for a long-running streaming job. Watermarking lets Spark safely discard/finalize old window state once it's confident no more relevant data will arrive.

Trade-off: a longer watermark tolerates more late data (fewer dropped events) but keeps more state in memory longer (higher resource usage, higher latency before results are finalized).

Interview Q&A

Q: Why is event-time windowing generally preferred over processing-time windowing for business aggregations? A: Processing time reflects when Spark happened to handle the data — which is affected by network delays, backpressure, retries, and cluster scheduling, none of which are meaningful to the business question being asked. Event time reflects when the event actually happened in the real world (e.g., when the sensor reading was taken, when the order was placed), so aggregating by event time gives correct, reproducible results regardless of processing delays — "how many orders per 5-minute window" should mean the same thing whether the pipeline processed it in real time or caught up after an hour of downtime.

Q: What problem does watermarking solve, in plain terms? A: Without any bound on how late data can arrive, Spark would have to keep every historical time window's aggregation state open forever "just in case" a late event shows up for it — memory usage would grow unboundedly over the life of a long-running streaming job. Watermarking says "I'll tolerate late data up to N minutes late, but after that, I'll finalize and forget that window" — giving Spark a safe, explicit point at which to release state and stop waiting.

Q: If you set a watermark of 10 minutes and an event arrives 15 minutes late, what happens to it? A: It's dropped from that windowed aggregation — the window it belonged to has already been finalized and its state discarded, since it exceeded the watermark's late-arrival tolerance. This is an intentional trade-off:

some late data loss in exchange for bounded memory usage and timely finalized results. If losing that late data is unacceptable for your use case, you'd need a longer watermark (at the cost of more state/memory and later-finalized results) or a separate mechanism (e.g., a side output/dead-letter path) to capture and handle exceptionally late records outside the windowed aggregation.

Q: How would you choose the watermark duration for a real pipeline? A: Base it on the actual observed/expected lateness distribution of your source data — e.g., look at historical event-time vs. processing-time gaps (how late do events typically/rarely arrive), and pick a threshold that covers the vast majority of realistic lateness (e.g., covers p99 of observed delay) while keeping state size and result latency acceptable for the business need. It's a genuine trade-off decision, not a default you set-and-forget — too short drops legitimate late data, too long balloons state and delays finalized results.

7.4 Streaming joins, checkpointing, fault tolerance

Notes

Streaming joins:

Stream-static join: streaming DataFrame joined with a regular (batch/static) DataFrame — straightforward, the static side is (re-)read as needed and is not stateful.

Stream-stream join: both sides are streams — this is stateful and requires watermarks on both sides (to bound how long Spark waits for a match before giving up), since a matching event on the other stream could theoretically arrive at any time without a watermark bound.

Checkpointing: every streaming query must specify a checkpoint location (`.option("checkpointLocation", path)`) — Spark persists the query's progress (source offsets processed, aggregation state) here, so if the job crashes/restarts, it resumes exactly where it left off rather than reprocessing from scratch or skipping data.

Fault tolerance guarantee: Structured Streaming provides **exactly-once** processing semantics (end-to-end, when using replayable sources like Kafka/files and idempotent/transactional sinks like Delta Lake) — achieved via the combination of checkpointed offsets (so failed/restarted batches reprocess the *exact same* input) and idempotent writes to the sink (so reprocessing the same batch doesn't double-write output).

Changing a streaming query's logic (e.g., adding a new aggregation, changing the schema significantly) after a checkpoint already exists can break resumability — some changes are safe (adding a filter), others require starting a fresh checkpoint (which means losing the "resume exactly where I left off" property and potentially reprocessing or skipping data, depending on offset handling).

Interview Q&A

Q: Why do stream-stream joins require watermarks on both sides, while a stream-static join doesn't need one at all? A: In a stream-stream join, Spark must buffer/hold state from *both* streams to wait for a potential future matching row on the other side — without a watermark bound, it would have to hold that state indefinitely (unbounded state growth), since a match could theoretically arrive at any future point. Watermarks on both sides let Spark bound how long it waits before giving up on finding a match and discarding old, unmatched state. A stream-static join has no such problem — the static side isn't accumulating unbounded new state over time, so there's no equivalent unbounded buffering concern.

Q: What exactly does the checkpoint location store, and why is losing it a serious problem? A: It stores the query's processed source offsets/positions (so Spark knows exactly what's already been consumed) and, for stateful operations (aggregations, joins), the accumulated intermediate state. Losing the checkpoint means the query loses track of what's already been processed — restarting it would either

reprocess data from scratch (duplicates, if the sink isn't also protecting against that) or, if configured to start from "latest," silently skip whatever arrived while the checkpoint was lost, both of which break correctness guarantees.

Q: How does Structured Streaming achieve "exactly-once" semantics when the underlying mechanics (checkpointed offsets + batch reprocessing on failure) sound like they could produce duplicates? A: The checkpointed offsets guarantee that if a batch fails and is retried, Spark reprocesses the *exact same* input data (same offset range) — not different data, not skipped data. That alone would give "at-least-once" (safe replays, but could double-write if a batch partially wrote and then retried). True exactly-once requires the *sink* to also be idempotent/transactional for that replayed write — e.g., Delta Lake's ACID transactions ensure a retried batch's write either fully lands once or the retry is a genuine no-op/overwrite of the same result, rather than appending a duplicate. So "exactly-once" is really "at-least-once processing + idempotent sink" working together.

Q: If you need to change the aggregation logic in an already-running streaming job, what's the risk? A: Structured Streaming's checkpoint stores state tied to the specific query plan (e.g., which columns are grouped, what aggregation functions are used) — changing that logic can make the existing checkpoint incompatible, since the saved state no longer matches the new plan's expectations. Spark will often explicitly fail to restart with an incompatible checkpoint for such changes. The typical resolution is to start a new checkpoint location (losing exact resume position — you'd need a plan for how to handle the gap/transition, e.g. reprocessing from a known offset) rather than assuming you can hot-swap logic under an existing checkpoint.

Module 8 — Production & Orchestration

8.1 Cluster sizing & configuration tuning (executor memory/cores)

Notes

Key configs: `spark.executor.memory` (heap per executor), `spark.executor.cores` (cores per executor, = max concurrent tasks per executor), `spark.executor.instances` (number of executors), `spark.driver.memory`, `spark.executor.memoryOverhead` (off-heap memory for JVM overhead, native libraries, etc — defaults to a fraction of executor memory, bump it up if you see "container killed by YARN for exceeding memory limits" type errors).

Sizing philosophy: **more, smaller executors** (e.g., 4-5 cores each) is generally recommended over few giant executors — very large executors (many cores) can suffer from JVM garbage collection pauses affecting many concurrent tasks at once, and from HDFS/network throughput limits per executor. But too many tiny executors adds per-executor overhead (each has its own JVM, connection overhead) and reduces the memory available per task for things like broadcast joins.

A common rule of thumb starting point: 4-5 cores per executor, leave 1 core per node for OS/daemon overhead, size executor memory based on $(\text{node memory} - \text{overhead}) / (\text{executors per node})$.

`spark.sql.shuffle.partitions` (see 5.2) should generally be tuned relative to cluster size/data volume — with AQE enabled (default on in Spark 3.x+), Spark can auto-coalesce this at runtime, reducing manual tuning need.

Dynamic allocation (`spark.dynamicAllocation.enabled=true`) lets Spark scale the number of executors up/down during a job based on actual workload, instead of a fixed count for the whole job — useful for variable workloads/shared clusters, avoiding over-provisioning idle executors.

Interview Q&A

Q: Why is "many small executors" generally preferred over "few large executors" with the same total resources? A: Very large executors (many cores, huge heap) tend to suffer more from JVM garbage collection pauses that affect all concurrently running tasks on that executor at once, and can hit per-executor I/O/network throughput ceilings (e.g., HDFS client throughput limits) that become a bottleneck when many tasks on one executor compete for it. Moderately-sized executors (commonly cited: 4-5 cores) balance parallelism, GC behavior, and I/O throughput better in most workloads. That said, too many tiny executors adds JVM startup/overhead cost and per-executor memory fragmentation, so it's a balance, not "smaller is always better."

Q: What does `spark.executor.memoryOverhead` cover, and when would you need to increase it? A: It's off-heap memory reserved for JVM overhead beyond the heap itself — things like native library allocations (e.g., Python worker processes for UDFs, PyArrow), JVM internal structures, NIO buffers. You'd increase it if you see the job killed by the cluster manager (YARN/Kubernetes) for exceeding its memory container limit despite `spark.executor.memory` seemingly being sufficient — a classic symptom when using PySpark UDFs/Pandas UDFs heavily, since Python worker processes consume memory outside the JVM heap that this setting needs to account for.

Q: What's dynamic allocation, and what's a scenario where you'd want it off instead? A: Dynamic allocation lets Spark request more executors when there's a backlog of pending tasks and release executors when they're idle, rather than holding a fixed number for the whole job's duration — great for variable/bursty workloads on a shared cluster, avoiding wasted idle resources. You might disable it for a job with tightly predictable, uniform resource needs where the overhead/latency of scaling executors up and down mid-job (and potential loss of cached data on released executors) isn't worth it, or in environments where a fixed, guaranteed resource allocation is required for predictable SLAs.

Q: How would you go about diagnosing whether a slow job is CPU-bound, memory-bound (spilling), or I/O-bound? A: Check the Spark UI: high **shuffle spill** metrics (memory and disk spill columns on the stages tab) point to memory pressure — not enough execution memory for the shuffle/aggregation, forcing disk spills. Long **task duration with low CPU utilization** on the executors tab suggests I/O-bound (waiting on network/disk reads, e.g., a slow source system or under-parallelized read). High CPU utilization with reasonable task durations and no spill suggests genuinely CPU-bound work (heavy computation, possibly a Python UDF), where the fix is more about algorithm/parallelism than memory tuning.

8.2 Job orchestration (Airflow / Databricks Workflows)

Notes

Orchestration tools handle: scheduling (cron-like or event-triggered), dependency management (job B only runs after job A succeeds), retries on failure, alerting, and often parameterization across environments (dev/staging/prod).

Airflow: DAGs defined in Python, tasks represent units of work (often a `SparkSubmitOperator`, or a `DatabricksSubmitRunOperator`/`DatabricksRunNowOperator` for Databricks-specific jobs), with rich scheduling/retry/alerting configuration and a large ecosystem of integrations. General-purpose, works across many tools beyond just Spark.

Databricks Workflows: native Databricks job scheduler, tightly integrated with Databricks clusters/notebooks, simpler to set up for Databricks-only pipelines, supports task dependency graphs, retries, and cluster reuse across tasks in a job. Less flexible for orchestrating *non*-Databricks systems compared to Airflow.

Common pattern: Airflow orchestrates the overall cross-system pipeline (e.g., "wait for file, trigger Spark job on Databricks, then trigger a downstream dbt run, then notify"), delegating the actual Spark execution to Databricks Workflows or a direct job submission.

Idempotency (Module 6.1) and good failure/retry design matter enormously here — orchestrators will retry failed tasks, so the underlying Spark jobs must tolerate reruns safely.

Interview Q&A

Q: When would you choose Airflow over Databricks Workflows for orchestrating a Spark pipeline, or vice versa? A: Choose Airflow when the pipeline spans multiple systems beyond Spark (e.g., needs to wait on an external API, trigger a dbt run, coordinate with non-Databricks infrastructure, or you need a single pane of glass across many different tech stacks in the org). Choose Databricks Workflows when the pipeline lives entirely within Databricks and you want simpler setup, tighter integration (e.g., shared cluster reuse across tasks, native notebook/job UI), and less operational overhead than running/maintaining a separate Airflow deployment.

Q: How does an orchestrator's retry mechanism interact with pipeline idempotency, and why does that matter? A: When a task fails, orchestrators typically retry it automatically (with configurable retry counts/backoff) — if the underlying job isn't idempotent, that automatic retry can silently corrupt data (e.g., duplicate appends) rather than safely resuming/redoing the work. This is why idempotent pipeline design (Module 6.1) isn't just a nice-to-have — it's a prerequisite for orchestrator-driven retries to be safe at all.

Q: What's a sensible way to model dependencies between a "raw ingestion" job and a "transform to Silver" job in an orchestrator? A: Model them as separate tasks with an explicit dependency edge (transform task depends on/downstream of ingestion task succeeding) — the orchestrator won't trigger the transform until ingestion reports success. You'd typically also want the transform task to validate that expected data actually landed (not just that the ingestion task "succeeded" in a shallow sense) before proceeding, to catch cases like partial/empty successful-looking runs.

8.3 Monitoring via Spark UI

Notes

Jobs tab: overview of all jobs (triggered by actions) in the application, their stages, and duration — good starting point to see which action/stage is the bottleneck.

Stages tab: per-stage breakdown — task count, duration distribution (this is where you spot **skew**: look at task duration min/median/max spread), shuffle read/write volume, and **spill** metrics (memory/disk spill = memory pressure indicator).

Storage tab: shows cached/persisted RDDs/DataFrames, their storage level, size in memory/disk, and fraction cached — useful for verifying caching is actually working as intended and checking for cache eviction issues.

Executors tab: per-executor resource usage — active tasks, memory used (storage vs. execution), shuffle read/write, task time, and importantly **GC time** — high GC time relative to task time indicates memory pressure/inefficient memory usage.

SQL/DataFrame tab: visual representation of the query plan (mirrors `.explain()` output) with actual runtime metrics annotated on each node (rows processed, time spent) — extremely useful for pinpointing exactly which operator in a complex query is the bottleneck, rather than guessing from the static plan alone.

Practical debugging flow: Jobs tab (which job is slow) → Stages tab (which stage, check for skew/spill) → SQL tab (which specific operator within that stage) → Executors tab (is it a resource/GC problem across the board, or isolated).

Interview Q&A

Q: You're told "the Spark job is slow" with no other details. Walk through how you'd use the Spark UI to diagnose it. A: Start at the **Jobs tab** to see which job/stage is taking disproportionately long. Drill into that job's **Stages tab** and look at the task duration distribution — a big spread between median and max task duration signals **skew**; check shuffle read size per task for confirmation (skewed tasks usually have much larger shuffle read). Check the **spill** columns — significant memory/disk spill indicates memory pressure, suggesting insufficient executor memory or too few shuffle partitions for the data volume. Check the **SQL tab** for that query to see the annotated plan with real row counts/timings per operator, pinpointing exactly which join/aggregation/scan is expensive. Finally check the **Executors tab** for GC time — high GC time across many executors points to memory configuration issues rather than a single bad operator.

Q: What does high GC time on the Executors tab typically indicate, and what would you try to fix it?

A: It indicates the JVM is spending significant time doing garbage collection relative to actual task execution — usually a sign of memory pressure: too many objects being created and short-lived (common with certain UDF patterns or excessive object allocation), insufficient executor memory for the workload, or too much data cached without adequate storage memory fraction. Fixes: increase executor memory, reduce the number of cores per executor (fewer concurrent tasks competing for the same heap, reducing allocation pressure), reduce caching footprint, or address the root cause if it's UDF-driven object churn (e.g., switch to a Pandas UDF or built-in function).

Q: How would you use the Storage tab to verify your caching strategy is actually helping, not hurting?

A: Check the "Fraction Cached" — if it's less than 100%, some partitions aren't fully cached (spilled to disk or evicted), meaning you're not getting the full benefit and might be silently paying recomputation cost for the missing fraction. Check the storage level actually being used matches what you intended, and check total size against available cluster memory — if cached data is crowding out execution memory for other operations (visible via increased spill elsewhere), that's a sign you're over-caching relative to available resources.

8.4 CI/CD for Spark jobs, testing with pytest + chispa/pyspark-test

Notes

Unit testing Spark transformations: structure code so transformation logic is in plain functions taking/returning DataFrames (not tangled inside a monolithic `main()`), so tests can call them directly with small, controlled input DataFrames and assert on output — no need to spin up a full cluster, a local

`SparkSession` (`master("local[*"])`) is enough for unit tests.



python

```
def clean_customers(df):
    return df.filter(F.col("email").isNull()).withColumn("email", F.lower("email"))

# test_clean_customers.py
def test_clean_customers(spark):
    input_df = spark.createDataFrame([("A@x.com"), (None,)], ["email"])
    result = clean_customers(input_df)
    assert result.count() == 1
```

chispa (and similarly `pyspark-test`) provide DataFrame equality assertion helpers designed for Spark's quirks — e.g., `assert_df_equality(actual_df, expected_df, ignore_row_order=True)` — since naive `==` comparison on DataFrames doesn't work the way you'd expect (partition/row order isn't guaranteed, comparing DataFrame objects directly doesn't do content comparison), and manually collecting + comparing lists is verbose and misses good error messages on mismatch.

A shared `pytest` fixture typically provides one `SparkSession` per test session (not per test — session creation is expensive) via `conftest.py`.

CI/CD pipeline structure: typically — lint/format check → unit tests (fast, local `SparkSession`, no real cluster) → build/package (e.g., wheel file or JAR) → deploy to target environment (upload to Databricks workspace/DBFS, register as a job) → optionally run smoke/integration tests against a real (small) cluster or staging environment before promoting to production.

Interview Q&A

Q: Why can't you just use Python's `==` to compare two Spark DataFrames in a test? A: `==` on DataFrame objects doesn't do a content/value comparison — comparing columns with `==` builds a *new Column expression* (used inside `.filter()`, e.g. `df.col1 == df.col2`), not a boolean result about DataFrame equality. Even collecting both to lists and comparing has issues: Spark doesn't guarantee row order across partitions unless you explicitly sort, so equally "correct" outputs can differ in row order and fail a naive comparison. Libraries like `chispa`'s `assert_df_equality` handle content comparison properly (with options like ignoring row order) and give useful diffs on failure.

Q: Why use a session-scoped `SparkSession` fixture instead of creating a new one per test function? A: Creating a `SparkSession` has real startup overhead (JVM initialization, etc.) — creating and tearing one down for every single test function would make a test suite with hundreds of tests painfully slow. A session-scoped fixture creates it once for the whole test run and reuses it across all tests, which is safe since `SparkSession` itself is generally stateless enough between independent DataFrame operations (each test creates its own fresh input DataFrames).

Q: How do you structure Spark code to make it actually testable, versus a script that's hard to unit test? A: Extract business logic into plain, pure-ish functions that take a DataFrame (and maybe simple parameters) as input and return a DataFrame as output — avoiding side effects like reading/writing to disk *inside* the transformation logic itself. I/O (reading sources, writing sinks) stays in a thin orchestration layer (e.g., a `main()` or job entrypoint) that calls these testable functions, so tests can exercise the transformation logic directly with small in-memory DataFrames without needing real files/tables/clusters.

Q: What would a reasonable CI pipeline stage sequence look like for a Spark project? A: Lint/static analysis (e.g., `flake8/black/mypy`) → fast unit tests against transformation logic using a local `SparkSession` → package the code (wheel/JAR) → deploy artifact to the target environment (e.g., upload to Databricks workspace) → run a smaller-scale integration/smoke test against real infrastructure (a real cluster, real-ish

sample data) before the deployment is promoted/considered complete, catching issues unit tests with toy data wouldn't (e.g., cluster config issues, real data edge cases, permission problems).

8.5 Cost optimization on cloud platforms

Notes

Spot/preemptible instances: significantly cheaper than on-demand, good fit for fault-tolerant Spark workloads (Spark can recompute lost partitions from lineage if a spot node is reclaimed) — commonly used for executor nodes (driver often kept on-demand for stability, since losing the driver kills the whole job).

Right-sizing clusters: avoid over-provisioning "just in case" — use dynamic allocation and monitor actual utilization (CPU/memory) rather than guessing; oversized clusters waste money on idle capacity.

Autoscaling + job-scoped (ephemeral) clusters: spin up a cluster for a job's duration and tear it down after, rather than paying for an always-on cluster sitting idle between scheduled runs — especially relevant for batch jobs that only run a few times a day.

File format/compression choices (Module 3.1) directly affect cost: efficient columnar formats with good compression reduce both storage cost and the compute/time (hence cost) needed to scan data.

Partition pruning / predicate pushdown reduce the actual data scanned per query — directly reduces compute time and, on platforms billing by data scanned (e.g., some serverless/warehouse-style billing) or compute-time, reduces cost proportionally.

Avoid unnecessary shuffles/caching — both cost compute time (hence cluster-hours, hence money); tuning per earlier modules (broadcast joins, right-sized shuffle partitions, deliberate caching) is directly cost-relevant, not just a "performance nicety."

Monitoring/cost visibility: tagging jobs/clusters for cost attribution, tracking cost per pipeline over time to catch regressions (e.g., a code change that accidentally introduced a full shuffle where a broadcast join used to apply).

Interview Q&A

Q: Why are spot/preemptible instances a good fit for Spark executor nodes specifically, but riskier for the driver? A: Spark's fault tolerance model means losing an executor (and its cached/in-progress data) is recoverable — the driver simply reschedules lost tasks on other executors and recomputes lost partitions from lineage, so a reclaimed spot executor causes a delay, not a failure. Losing the **driver**, however, kills the entire application outright (it holds the application's coordinating state) — so keeping the driver on stable on-demand infrastructure while using cheaper spot instances for executors is a common cost-optimization pattern that doesn't sacrifice job reliability.

Q: How does using ephemeral, job-scoped clusters versus a single always-on cluster affect cost for a set of daily batch jobs? A: An always-on cluster incurs cost 24/7 whether or not jobs are actively running — for jobs that run a few times a day and finish in an hour or two, most of that cost is pure waste from idle capacity. Ephemeral clusters (spun up specifically for a job run, torn down after) only incur cost for the actual compute time used — at the expense of some cluster startup latency per job (a real but usually acceptable trade-off for scheduled batch jobs, less acceptable for latency-sensitive interactive/ad hoc workloads where an always-on or fast-start cluster matters more).

Q: Give an example of how a performance optimization from earlier modules directly translates into a cost saving. A: Switching a large-small join from a sort-merge join to a broadcast join (Module 5.4) eliminates the shuffle of the large table — less compute time, less network/disk I/O, meaning the job

finishes faster and needs fewer cluster-hours, which is a direct, measurable cost reduction on cloud platforms billed by compute time/cluster uptime, not just a "nice to have" speed improvement.

Q: What's a cost-optimization risk specific to autoscaling clusters if not configured carefully? A: If scale-up/scale-down thresholds and cooldown periods aren't tuned well, a cluster can thrash — scaling up for a brief spike then scaling back down repeatedly, incurring the overhead cost of provisioning/deprovisioning nodes frequently without settling into a stable, efficient state — sometimes ending up *more* expensive and less performant than a well-sized fixed cluster for predictable, steady workloads. Autoscaling is most beneficial for genuinely variable/bursty workloads, not a blanket default for every job type.

Module 9 — Advanced / Interview-Ready Topics

9.1 Custom partitioners, custom data sources

Notes

Custom partitioners (RDD-level, `partitionBy(numPartitions, partitionFunc)`) let you control exactly how keys map to partitions instead of Spark's default hash partitioning — useful when you know something about your key distribution that lets you co-locate related data more intelligently than a generic hash would (e.g., ensuring all data for a specific range of IDs, or a specific composite grouping, lands on the same partition to optimize a specific downstream algorithm).

At the DataFrame level, direct custom partitioner control is more limited than RDDs (DataFrames use `repartition`/`partitionBy` on write, which doesn't expose the same low-level partitioning function hook) — this is one of the genuine cases (Module 1.5) where dropping to the RDD API buys you something DataFrames don't expose.

Custom data sources: Spark's Data Source API (V2 in modern Spark) lets you implement connectors for systems Spark doesn't natively support — implementing read/write logic, partition/split planning, and pushdown capability (filter/column pushdown) for a custom source. Real-world example: a company might build a custom data source connector for a proprietary internal system or a niche external API, exposing it to Spark users as `spark.read.format("mycustomsource").load(...)`.

Interview Q&A

Q: Give a concrete scenario where a custom partitioner would improve performance over default hash partitioning. A: If you're about to do many operations that repeatedly join or group by a composite/derived key (e.g., a combination of region and a hashed customer segment) that default hash partitioning doesn't handle optimally for your specific access pattern, a custom partitioner can ensure related data is co-located from the start, avoiding repeated re-shuffling across multiple downstream operations that all key on that same logical grouping.

Q: What's the difference between the Data Source V1 and V2 APIs, at a conceptual level, and why does it matter? A: V1 was simpler but limited — weaker support for pushdown optimizations (filter/column pushdown), less control over partitioning/read planning, and no native support for things like streaming sources or catalog integration in a unified way. V2 provides a much richer, more capable interface for source implementers — enabling proper pushdown, better partition planning control, unified batch/streaming source support, and catalog plugin support — meaning a well-implemented V2 source can perform much closer to Spark's native sources (Parquet, etc.) than a V1 source typically could.

Q: Why would you build a custom Spark data source instead of just reading data into a DataFrame with `createDataFrame` from a pre-fetched Python collection? A: `createDataFrame` from Python-fetched data means the fetch itself is a single-threaded, non-distributed operation on the driver before Spark even

starts distributed processing — this doesn't scale for large data volumes and creates a driver memory bottleneck. A proper custom data source implements distributed reading (partition/split planning so multiple executors read different chunks in parallel directly from the source), plus pushdown support, giving genuinely scalable, Spark-native ingestion rather than a driver-bottlenecked workaround.

9.2 Adaptive Query Execution (AQE)

Notes

AQE (Spark 3.x+, enabled by default from 3.2 onward) re-optimizes the query plan **at runtime**, using actual statistics gathered from completed shuffle stages, instead of relying solely on static, pre-execution estimates (which can be wrong due to stale/missing table statistics, complex filter selectivity, UDF outputs, etc.).

Three main AQE features:

Dynamically coalescing shuffle partitions: after a shuffle, if actual partition sizes are much smaller than expected (or wildly uneven), AQE merges small adjacent partitions into fewer, better-sized ones — reduces the "too many tiny tasks" overhead problem without needing to manually tune

`spark.sql.shuffle.partitions` perfectly upfront.

Dynamically switching join strategies: if a table turns out to be smaller than expected after filtering (observed from actual runtime shuffle stats), AQE can convert a planned sort-merge join into a broadcast join mid-query.

Dynamically optimizing skewed joins: detects skewed partitions from actual runtime shuffle statistics and splits them into smaller sub-partitions handled by multiple tasks (an automatic version of manual salting, Module 5.6).

Config flags: `spark.sql.adaptive.enabled` (master switch), `spark.sql.adaptive.coalescePartitions.enabled`, `spark.sql.adaptive.skewJoin.enabled` — all default to true in modern Spark.

AQE re-optimizes plans at **shuffle stage boundaries** specifically — it can only act on information available once a stage's shuffle output is materialized, so it's not a fully continuous/every-operator re-optimization, but a series of checkpoints where the plan can adapt.

Interview Q&A

Q: Why can't Spark's static (pre-execution) cost-based optimizer always make the right decisions upfront, and how does AQE address that? A: Static optimization relies on table/column statistics gathered ahead of time (which can be stale, missing, or simply hard to estimate accurately through complex filters, UDFs, or multi-way joins) — the optimizer's cardinality/size estimates can be significantly wrong, leading to suboptimal choices like picking a sort-merge join when a broadcast would've been far cheaper, or choosing a shuffle partition count that doesn't match actual data volume. AQE re-evaluates and adjusts these decisions at runtime using *actual* materialized statistics from completed shuffle stages, correcting for cases where the upfront estimate was wrong.

Q: Does AQE eliminate the need to manually configure `spark.sql.shuffle.partitions`? A: It significantly reduces the need for precise manual tuning, since AQE's dynamic coalescing can merge overly-small partitions after the fact regardless of the initial shuffle partition count — but it doesn't make the setting entirely irrelevant; a wildly wrong initial value can still affect the granularity/parallelism of the initial shuffle write, so reasonable starting values still help, just with much less pressure to get it perfectly tuned.

Q: Can AQE's join strategy switching happen for any join in a query, at any point? A: No — it happens specifically at shuffle stage boundaries within the query, where actual materialized size statistics become

available after a stage completes. It can't retroactively change decisions already made and executed earlier in a way that would require re-running completed work; it applies to decisions about stages that haven't executed yet, informed by what's now known from stages that have.

9.3 Memory management deep dive (execution vs. storage memory)

Notes

Spark's unified memory manager (Spark 1.6+) divides executor JVM heap (governed by `spark.memory.fraction`, default 0.6 of usable heap after reserving some for Spark's internal use) into two regions:

Execution memory: used for computation — shuffles, joins, sorts, aggregations (building hash tables, sort buffers, etc.).

Storage memory: used for caching (`.cache()`/`.persist()`) and broadcast variables.

These two regions share a unified pool with a **soft boundary** (`spark.memory.storageFraction`, default 0.5 of the memory pool): either region can borrow space from the other if the other isn't using its share, but **execution memory can evict cached (storage) data if it needs more space** (execution takes priority — a running computation can't simply fail because cache is hogging memory; it'll evict cached blocks as needed), whereas storage memory cannot evict active execution memory usage (this would break a running computation, which unified memory management protects against).

Beyond the unified `spark.memory.fraction` pool: **reserved memory** (fixed overhead for Spark internals, not configurable, small ~300MB) and **user memory** (the remaining fraction, `1 - spark.memory.fraction`, used for user data structures/objects outside Spark's managed memory, e.g. objects created inside UDFs).

This design is why heavy caching can slow down (not just fail) an otherwise fine job — if execution needs to evict a lot of cached data to make room mid-computation, that eviction (and potential later recomputation of evicted-then-needed-again cached data) adds real overhead compared to a cleaner memory budget.

Interview Q&A

Q: Why can execution memory evict storage (cached) memory, but not vice versa? A: A running computation (e.g., an in-progress shuffle or join building a hash table) *must* have the memory it needs to complete correctly — it can't gracefully "pause" or fail just because cached data is occupying space; that would break the job. Cached data, in contrast, is a performance optimization, not a correctness requirement — losing a cached partition just means Spark recomputes it from lineage next time it's needed, which is safe (if suboptimal) rather than causing a job failure. So the design prioritizes correctness (execution) over the performance optimization (caching) when they compete for the same limited memory pool.

Q: You've cached several large DataFrames and now a join in your job is running unusually slowly, with high shuffle spill. What might be happening, memory-wise? A: The execution memory needed for the join's shuffle/hash-table building may be competing with the storage memory occupied by your cached DataFrames — if execution can't get enough space (even after evicting what cache space it can reasonably reclaim), it spills excess data to disk to proceed, which is much slower than in-memory processing. The fix is typically to unpersist DataFrames you no longer need before the memory-intensive stage, or reduce what you cache, freeing up more room for execution memory during the join.

Q: What's the practical difference between "user memory" and Spark's managed execution/storage memory pool? A: User memory is the portion of the executor JVM heap *outside* Spark's unified memory manager — it holds user-defined data structures created inside your own code (e.g., objects instantiated

inside a UDF, or large Python objects if relevant) that Spark itself doesn't track or manage as part of its execution/storage accounting. If your UDFs or custom logic allocate large amounts of memory here, it's invisible to Spark's memory management and can still cause OOM issues that aren't reflected in the execution/storage memory metrics you'd see in the Spark UI — a common blind spot when debugging memory issues in jobs with heavy custom logic.

9.4 Common data engineering interview problems solved in PySpark

This section is a grab-bag of classic PySpark interview coding problems, with the core approach for each — the kind of thing that gets asked as a live-coding or take-home exercise.

Problem: Find duplicate records based on a subset of columns.

```
python
dupes = df.groupBy("email").count().filter("count > 1")
duplicate_rows = df.join(dupes, "email", "left_semi")
```

Problem: Find the second-highest salary per department.

```
python
w = Window.partitionBy("department").orderBy(F.col("salary").desc())
result = df.withColumn("rn", F.dense_rank().over(w)).filter("rn = 2")
```

(Use `dense_rank` not `row_number` if ties at the top should both count as "1st" and the "2nd highest" should mean the next distinct value.)

Problem: Pivot data from long to wide format (e.g., monthly sales rows → one column per month).

```
python
df.groupBy("product").pivot("month").agg(F.sum("sales"))
```

Problem: Compute a running total / cumulative sum per group, ordered by date.

```
python
w = Window.partitionBy("account_id").orderBy("date").rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)
df.withColumn("running_balance", F.sum("amount").over(w))
```

Problem: Find gaps/missing dates in a time series per entity.

```
python
# Generate the full expected date range, then left_anti join against actual data
all_dates = spark.sql("SELECT explode(sequence(to_date('2024-01-01'), to_date('2024-12-31'), interval 1 day))")
expected = all_dates.crossJoin(df.select("entity_id").distinct())
missing = expected.join(df, ["entity_id", "date"], "left_anti")
```

Problem: Deduplicate, keeping only the latest record per key (based on a timestamp column).



```
python
w = Window.partitionBy("id").orderBy(F.col("updated_at").desc())
latest = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
```

Problem: Word count (the classic, still occasionally asked to check API fluency).



```
python
result = df.select(F.explode(F.split(F.col("text"), " ")).alias("word")) \
    .groupBy("word").count().orderBy(F.desc("count"))
```

Problem: Implement a sessionization (group events into sessions with a gap threshold, e.g., 30 min of inactivity = new session).



```
python
w = Window.partitionBy("user_id").orderBy("event_time")
df = df.withColumn("prev_time", F.lag("event_time").over(w))
df = df.withColumn(
    "new_session",
    F.when(
        (F.col("prev_time").isNull()) |
        (F.col("event_time").cast("long") - F.col("prev_time").cast("long") > 1800),
        1
    ).otherwise(0)
)
df = df.withColumn("session_id", F.sum("new_session").over(w.rowsBetween(Window.unboundedPreceding, 1)))
```

Interview Q&A (meta-level, about approaching these problems)

Q: In a live-coding interview, what should you clarify before writing any PySpark code for a data problem? A: Ask about data scale (does it actually matter for the approach, or is correctness the focus), whether ties/edge cases matter (e.g., "second highest" with duplicate top values — dense_rank vs row_number), null handling expectations, and whether they want a specific output format (aggregated single row vs. row-per-record). These clarifying questions also signal you're thinking about correctness and production-readiness, not just pattern-matching a memorized snippet.

Q: Why is window functions + `row_number()`/`dense_rank()` such a common building block across so many of these problems (dedup, top-N, latest-record, gap detection)? A: Because a huge fraction of "give me the Nth / latest / first / ranked row per group" problems reduce to the same underlying pattern: partition by the grouping key, order by the tiebreak/priority column, assign a row-level rank, then filter on that rank — it's a single, general tool that replaces what would otherwise require self-joins or multiple separate aggregation-then-join steps (both less efficient and more error-prone).

Q: What's a common mistake candidates make when solving the "find duplicates" or "dedupe" style problems? A: Using `distinct()` when the intent was actually "keep one row per key based on some priority" (e.g., latest by timestamp) — `distinct()` only removes fully identical rows and gives no control

over *which* duplicate survives when rows differ in non-key columns. The window-function + `row_number()` pattern (partition by key, order by the tiebreak column, filter rank = 1) is almost always the more correct and controllable approach when "duplicates" means "multiple versions of the same logical record," not "byte-identical rows."

Quick-Reference: Fast Answers to "Why" Questions Interviewers Love

Why DataFrames over RDDs? Catalyst + Tungsten optimization; RDDs are opaque to the optimizer.

Why avoid UDFs? Break Catalyst optimization + row-by-row JVM↔Python serialization cost. Use built-ins or Pandas UDFs instead.

Why does a join get slow? Likely a shuffle (sort-merge) instead of broadcast, or skew on the join key.

Why is my job "stuck" on one task? Data skew — check Stages tab for uneven task duration/shuffle-read.

Why use Parquet over CSV? Columnar + predicate pushdown + column pruning + splittable + embedded schema.

Why partition data on write? Enables partition pruning on read — skip whole directories that can't match a filter.

Why does `overwrite` wipe my whole table on a daily job? Static overwrite mode replaces the entire path; use dynamic partition overwrite mode.

Why does caching not always help? Only helps with reuse; single-use DataFrames gain nothing and waste memory.

Why exactly-once in Structured Streaming? Checkpointed offsets (replay-safe) + idempotent/transactional sink (Delta) together.

Why watermarking? Bounds unbounded state growth in streaming aggregations by defining how late is "too late."

Why AQE matters? Static plans rely on estimates that can be wrong; AQE corrects join strategy/partition sizing using real runtime stats.

Why is idempotency non-negotiable in production ETL? Orchestrators retry failed jobs automatically — non-idempotent jobs corrupt data on every retry.

How to Use This Guide

Go module by module in order — each builds on the last (fundamentals → DataFrame ops → I/O → SQL → performance → DE patterns → streaming → production → advanced). For interview prep specifically: Read the notes for a topic once.

Cover the notes, and try to answer each Q&A from memory in your own words.

Pay special attention to Module 5 (Performance & Internals) and Module 9 — these are where most "senior" data engineering interviews go deep, beyond just knowing the API.

For the coding problems in 9.4, actually type them out rather than just reading — muscle memory for window function syntax matters under interview time pressure.

Good luck — you're well covered on this roadmap.